

Introduction to Computer Vision



Lec.08: Depth estimation and 3D reconstruction
2023.11.13

Today

- Depth estimation
 - 1. Introduction**
 2. Stereo matching
 3. Multi-view stereo
- 3D reconstruction

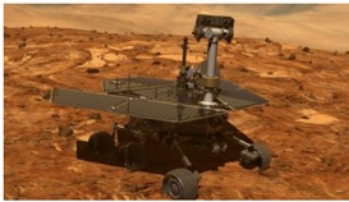
Depth sensing

- Compute depth maps



Depth sensing

- Depth is a crucial cue for many computer vision applications



Robotic (NASA)



Autonomous driving (Google)



Biometric (Apple)



Drones (DJI)

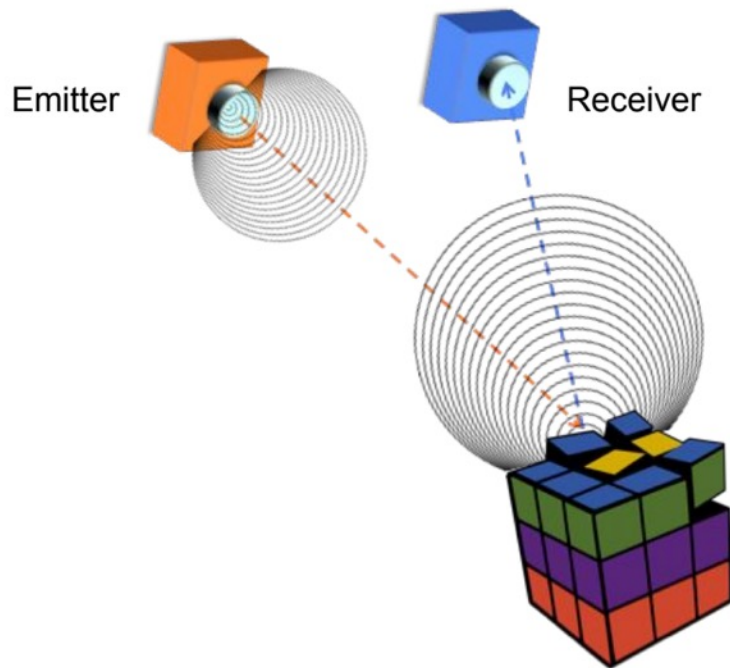


Gaming (Microsoft)



Augmented Reality (Microsoft)

Active depth sensing



Depth is perceived by perturbing the sensed environment:

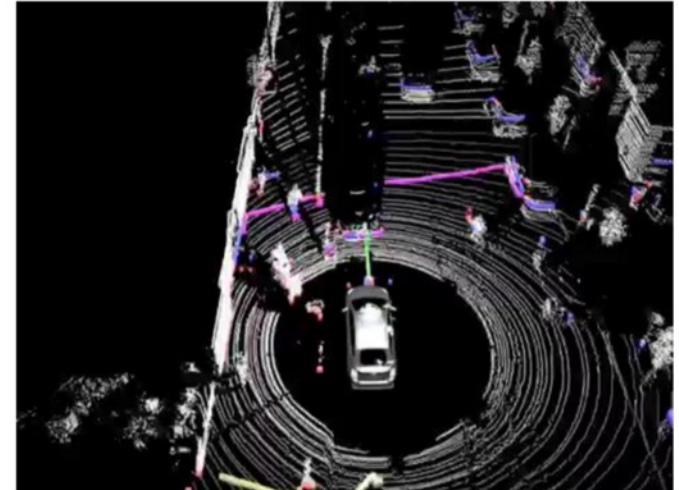
- LiDAR (e.g., Velodyne)
- Structured light (e.g., Kinect 1)
- Active stereo (e.g., Intel RealSense)

Active depth sensing

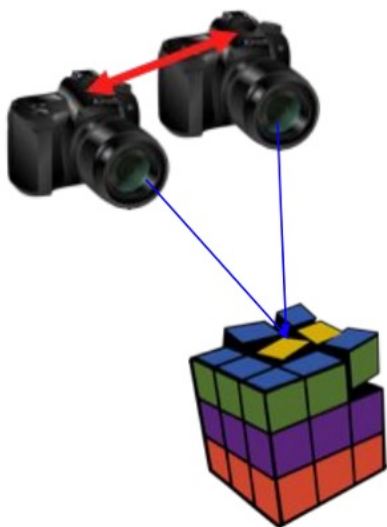
LiDAR: **L**ight **D**etection and **R**anging



- Based on Time of Flight (ToF)
- Expensive
- Extremely accurate depth information
- High resolution
- 360 degrees of visibility



Passive depth sensing



Stereo



Monocular

Today

- Depth estimation
 1. Introduction
 - 2. Stereo matching**
 3. Multi-view stereo
 4. Monocular depth estimation

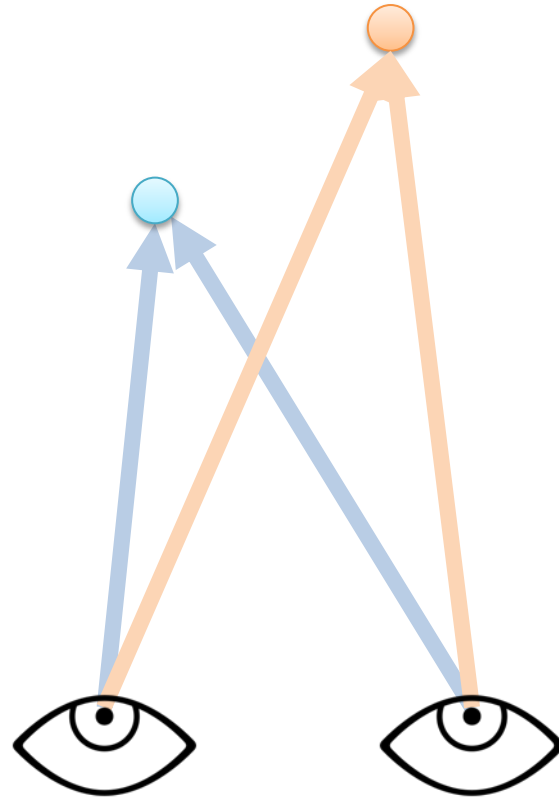
Stereo vision



<https://giphy.com/gifs/wigglegram-706pNfSKyaDug>

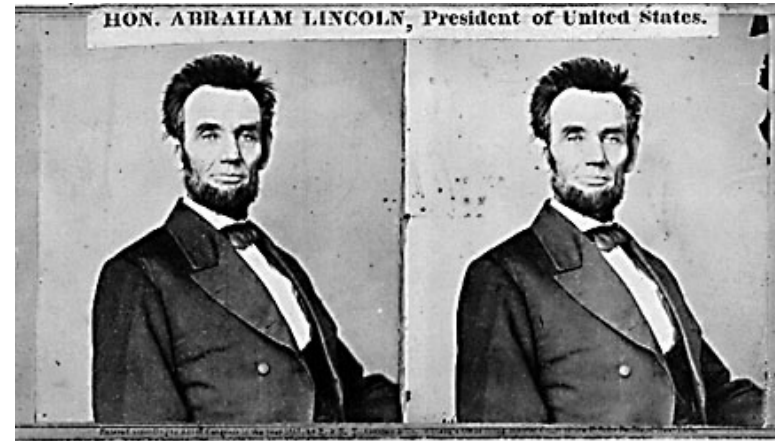
Stereo vision

- An object point will project to some point in our image
- That image point corresponds to a ray in the world
- Two rays intersect at a single point, so if we want to localize points in 3D we need 2 eyes



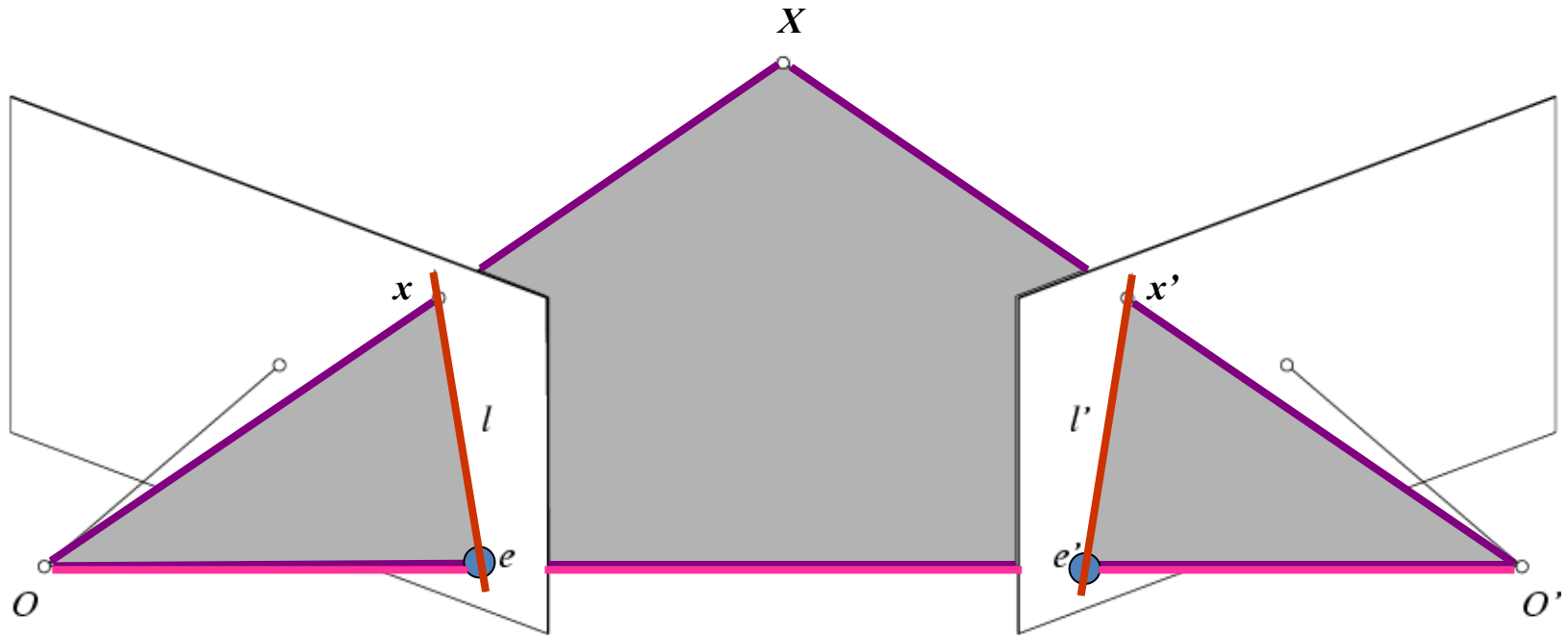
Stereo vision

- Suppose the relative camera pose is given, how to compute the depth of each point in the image?
 1. Find 2D-2D correspondences
 2. Triangulate
- **Stereo matching:** how to match all pixels efficiently?



Recap: Epipolar geometry

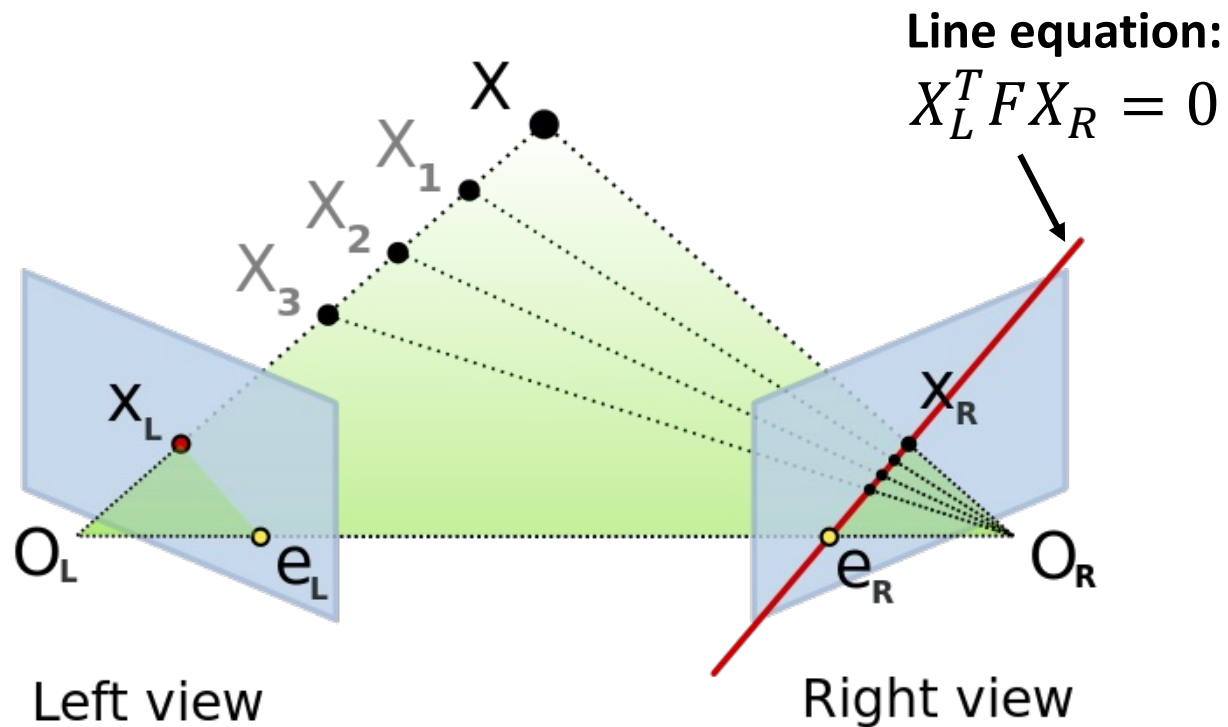
Recap: Epipolar geometry



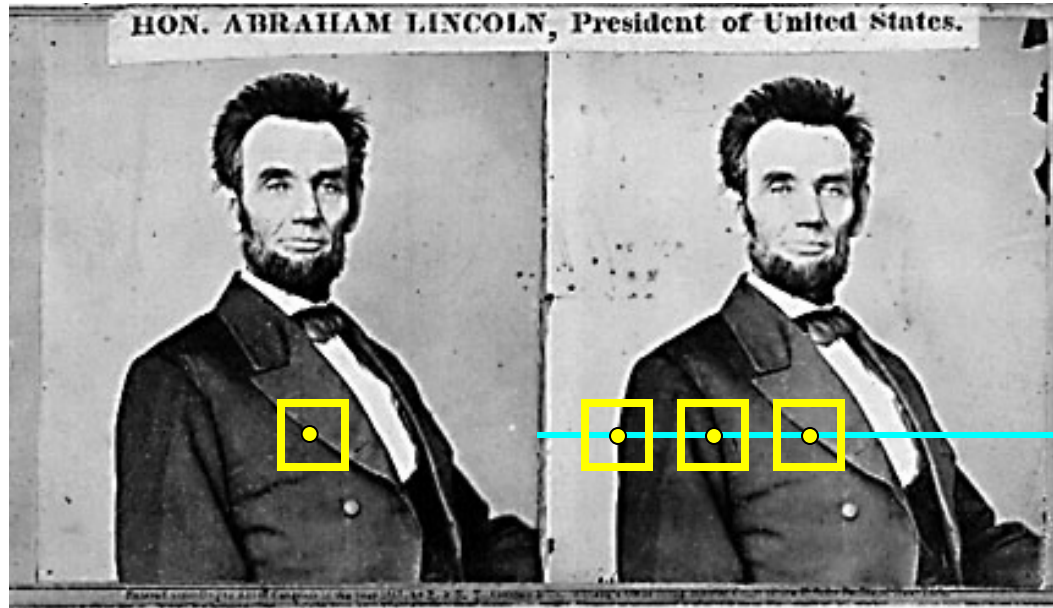
- **Baseline:** line connecting the two camera centers (OO')
- **Epipoles:** intersections of baseline with image planes (e and e')
- **Epipolar Plane** – plane containing (XOO')
- **Epipolar Lines:** intersections of epipolar plane with image planes (l and l')

Recap: Epipolar geometry

Given X_L , its match in the other image X_R must lie on the **epipolar line (the red line)**.



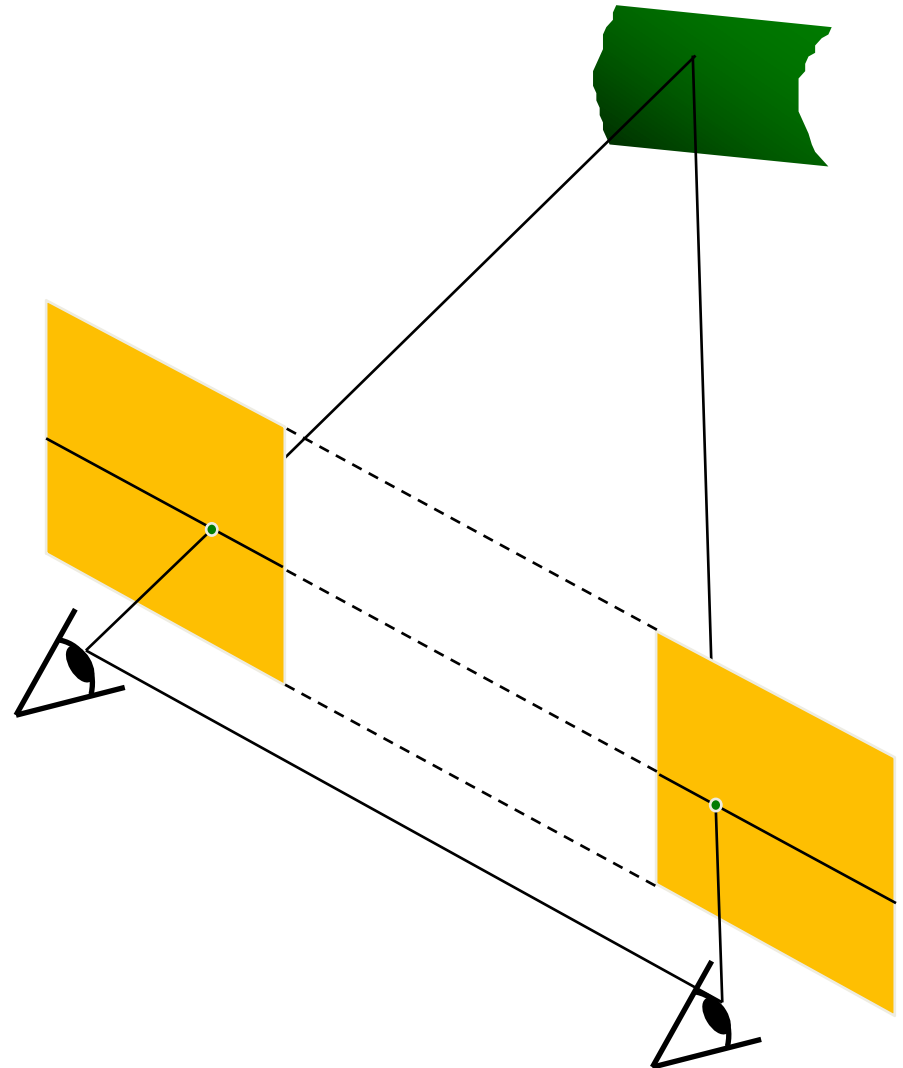
Basic stereo matching algorithm



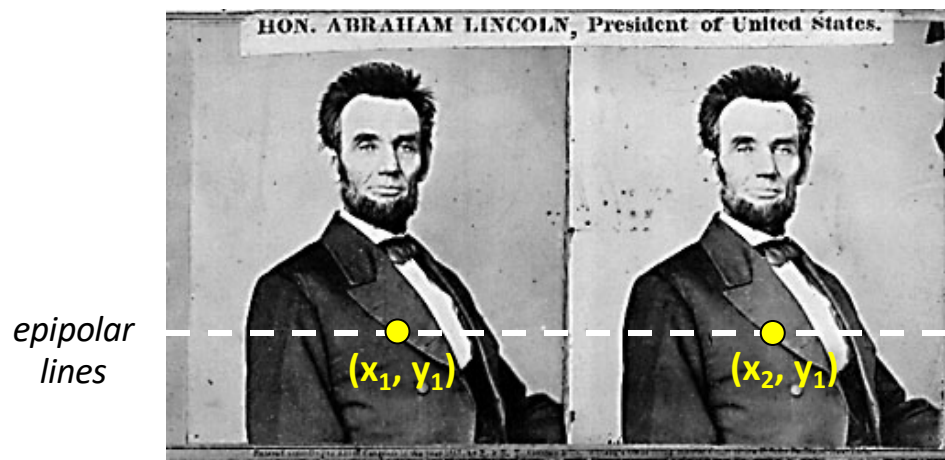
- For each pixel in the first image
 - Find corresponding epipolar line in the right image
 - Search along epipolar line and pick the best match
- Simplest case: epipolar lines are horizontal scanlines
 - When does this happen?

Simplest Case: Parallel images

- Image planes of cameras are parallel to each other and to the baseline
- Camera centers are at same height
- Focal lengths are the same
- Then, epipolar lines fall along the horizontal scan lines of the images



Depth from disparity

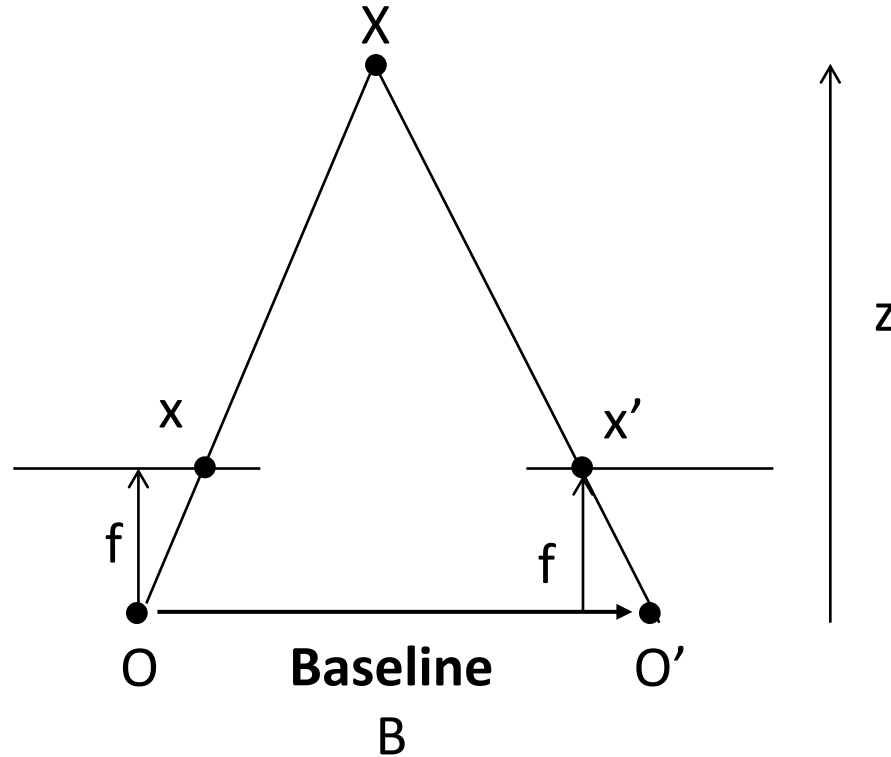


Two images captured by a purely horizontal translating camera
(*rectified* stereo pair)

$x_2 - x_1 =$ the *disparity* of pixel (x_1, y_1)

Depth from disparity

Note that x and x' are image coordinates relative to the image centers, respectively

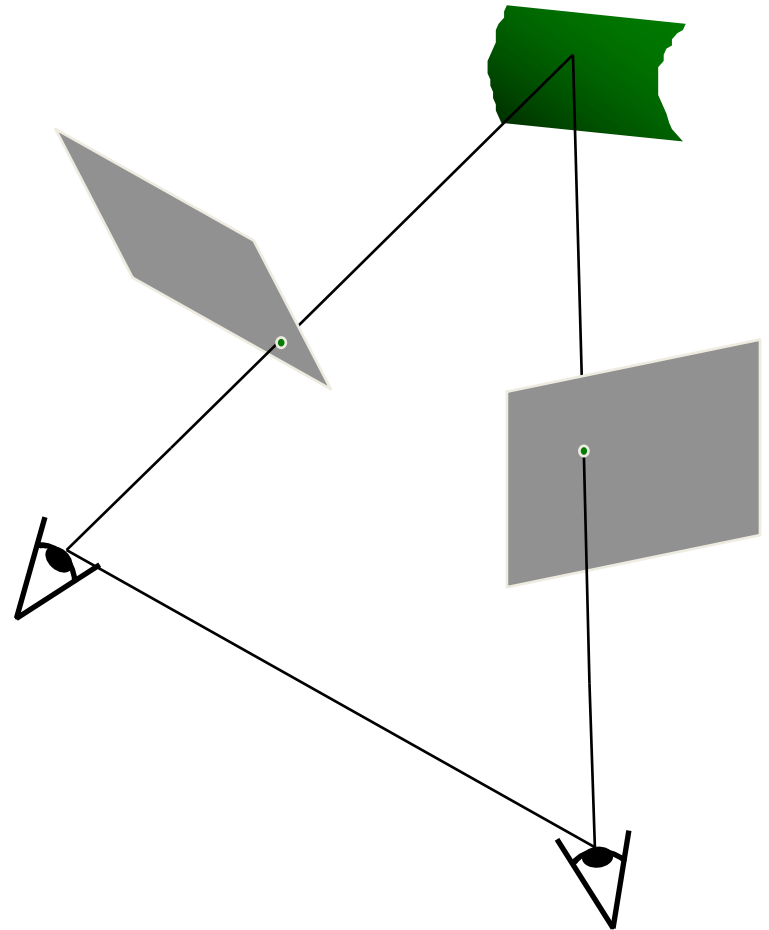


$$\text{disparity} = x - x' = \frac{B \cdot f}{z}$$

Disparity is inversely proportional to depth.

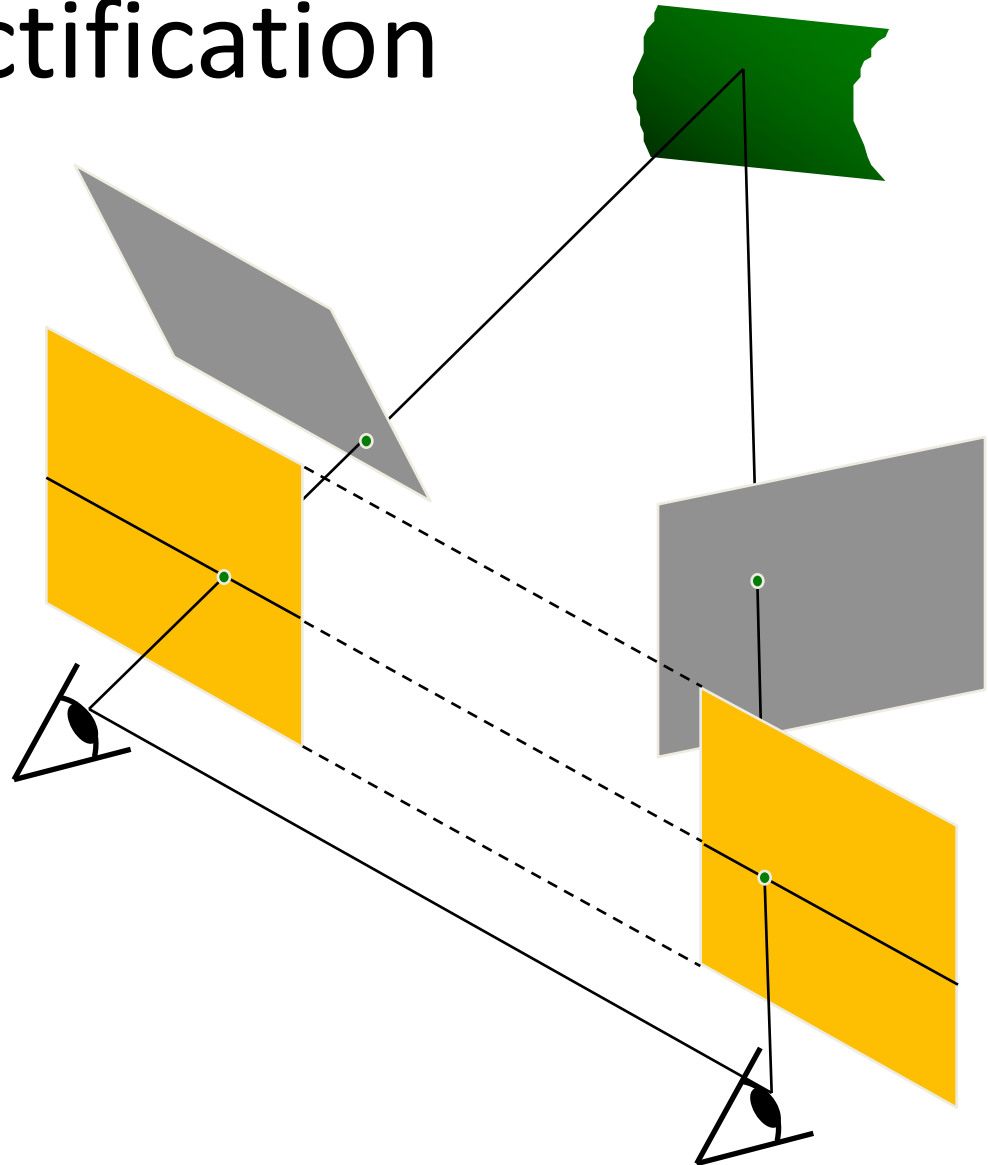
More complicated cases

- What can we do if epipolar lines are not scanlines?



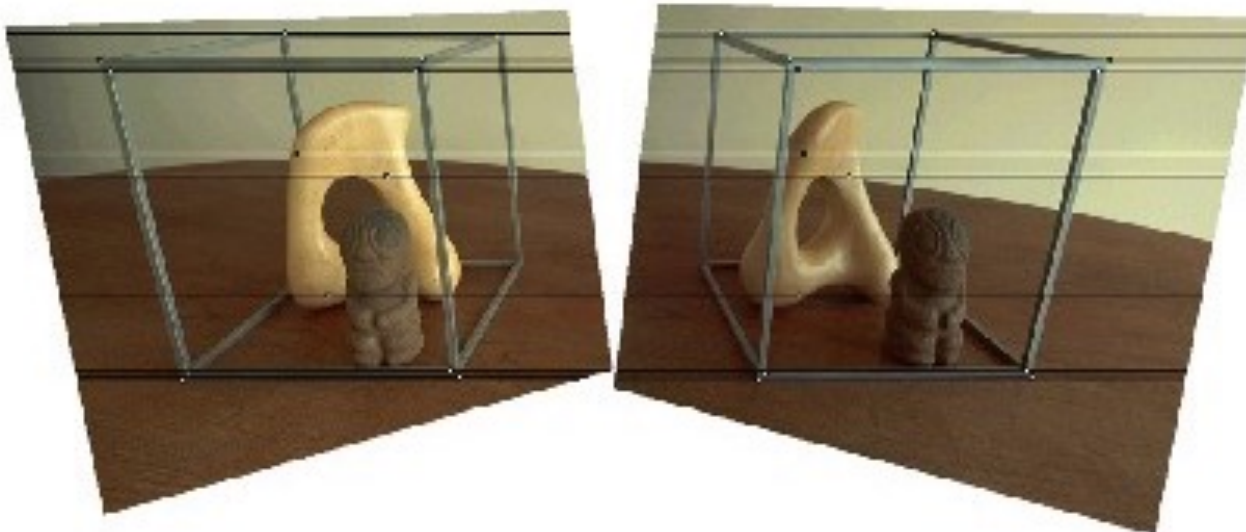
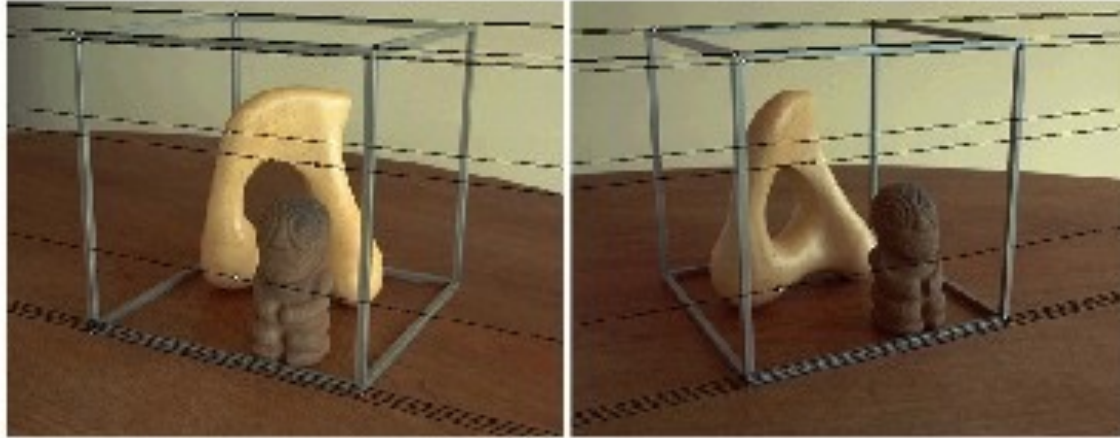
Stereo image rectification

- Reproject image planes onto a common plane parallel to the line between camera centers
- Two homographies (3x3 transform), one for each input image reprojection



C. Loop and Z. Zhang. [Computing Rectifying Homographies for Stereo Vision](#). IEEE Conf. Computer Vision and Pattern Recognition, 1999.

Rectification example



Stereo matching algorithms

- Match pixels in conjugate epipolar lines
 - Assume brightness constancy
 - This is a challenging problem
 - Hundreds of approaches
 - A good survey and evaluation:
<http://www.middlebury.edu/stereo/>

Basic stereo matching algorithm

- Best match minimizes the dissimilarity



Popular matching scores

- SSD (Sum of Squared Differences) $\sum_{x,y} |W_1(x,y) - W_2(x,y)|^2$
- SAD (Sum of Absolute Differences) $\sum_{x,y} |W_1(x,y) - W_2(x,y)|$
- ZNCC (Zero-mean Normalized Cross Correlation)

$$\frac{\sum_{x,y} (W_1(x,y) - \overline{W_1})(W_2(x,y) - \overline{W_2})}{\sigma_{W_1} \sigma_{W_2}}$$

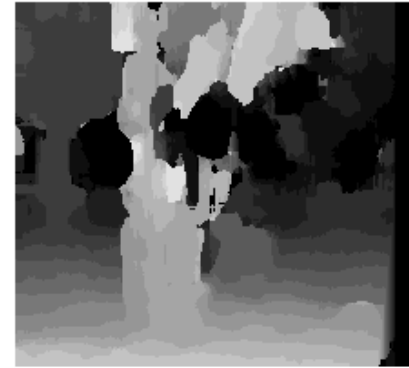
- where $\overline{W_i} = \frac{1}{n} \sum_{x,y} W_i$ $\sigma_{W_i} = \sqrt{\frac{1}{n} \sum_{x,y} (W_i - \overline{W_i})^2}$
- what advantages might NCC have?

Window size



$W = 3$

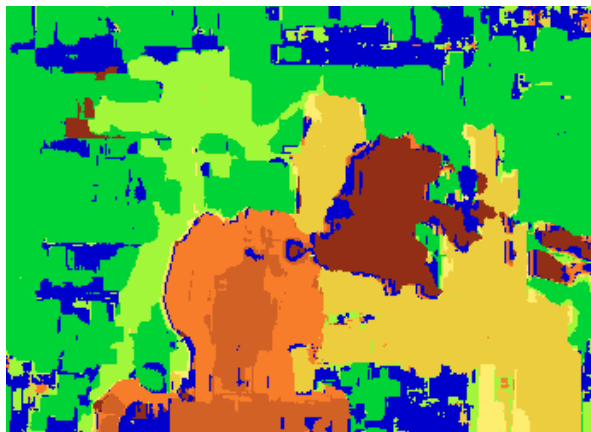
Smaller window
+ more detail
- more noise



$W = 20$

Larger window
+ less noise
- less detail

Results with window search



Window-based matching
(best window size)



Ground truth

Better methods exist...



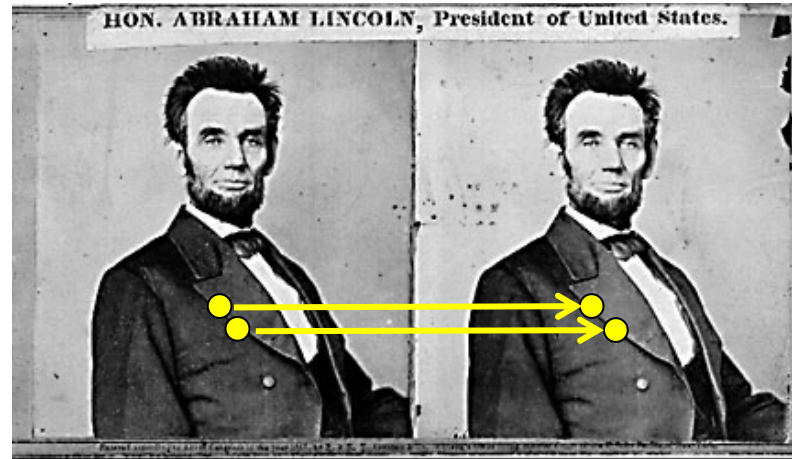
Graph cuts-based method

Boykov et al., [Fast Approximate Energy Minimization via Graph Cuts](#),
International Conference on Computer Vision 1999.



Ground truth

Stereo as energy minimization



$$E(d) = \underbrace{E_d(d)}_{\text{match cost}} + \underbrace{\lambda E_s(d)}_{\text{smoothness cost}}$$

Variables: disparity of each pixel

Want each pixel to find a good match in the other image

Adjacent pixels should (usually) move about the same amount

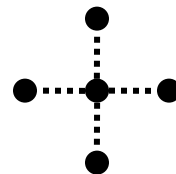
Stereo as energy minimization

$$E(d) = E_d(d) + \lambda E_s(d)$$

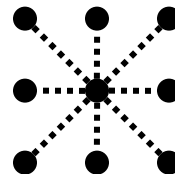
match cost: $E_d(d) = \sum_{(x,y) \in I} C(x, y, d(x, y))$ e.g., SSD, SAD, ZNCC

smoothness cost: $E_s(d) = \sum_{(p,q) \in \mathcal{E}} V(d_p, d_q)$

$\mathcal{E}$: set of neighboring pixels



4-connected
neighborhood



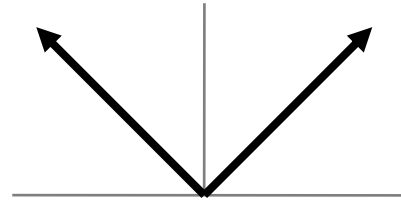
8-connected
neighborhood

Smoothness cost

How do we choose V ?

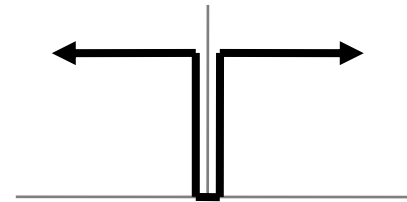
$$V(d_p, d_q) = |d_p - d_q|$$

L_1 distance



$$V(d_p, d_q) = \begin{cases} 0 & \text{if } d_p = d_q \\ 1 & \text{if } d_p \neq d_q \end{cases}$$

“Potts model”

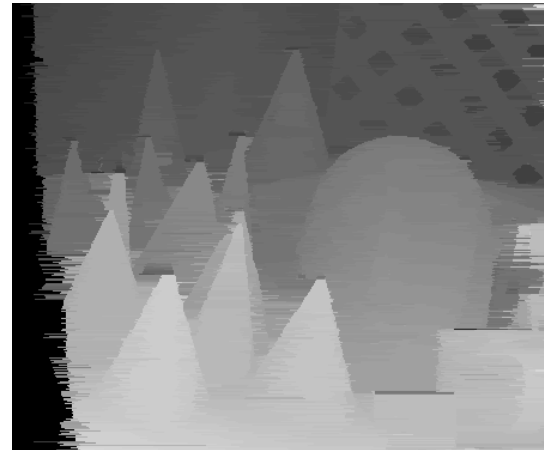


Dynamic programming

$$E(d) = E_d(d) + \lambda E_s(d)$$

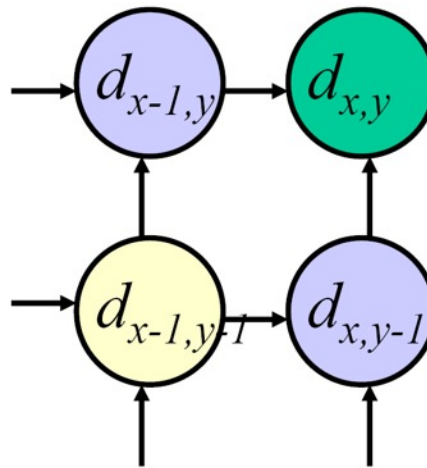
- Can minimize this independently per scanline using dynamic programming (DP)

Dynamic programming



Dynamic programming

- Can we apply DP in 2D as well?



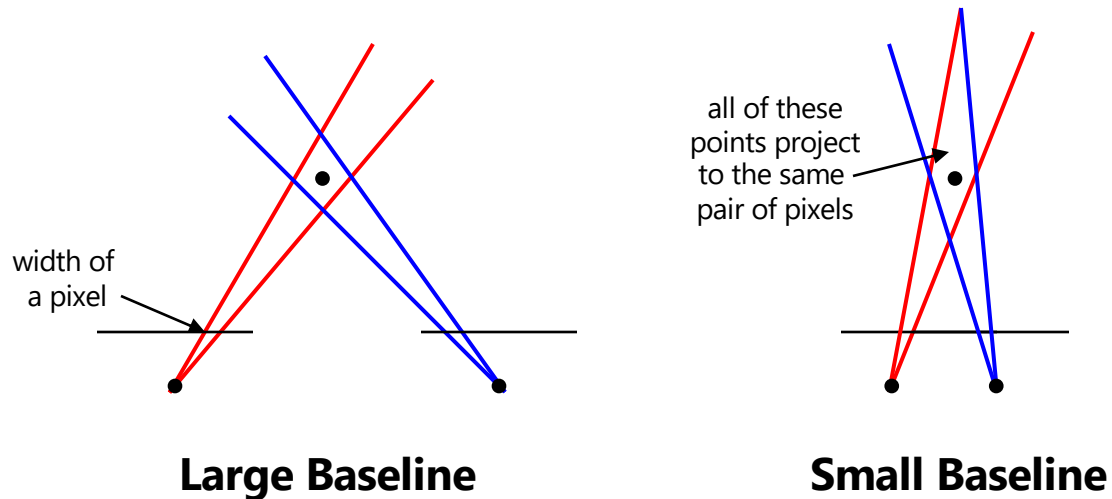
- No, DP only works to find a 1D path
- NP-hard, but can be approximately solved by graph cut or belief propagation algorithms

Stereo reconstruction pipeline

Steps

1. Calibrate cameras
2. Rectify images
3. Compute disparity
4. Estimate depth

Choosing the stereo baseline



What's the optimal baseline?

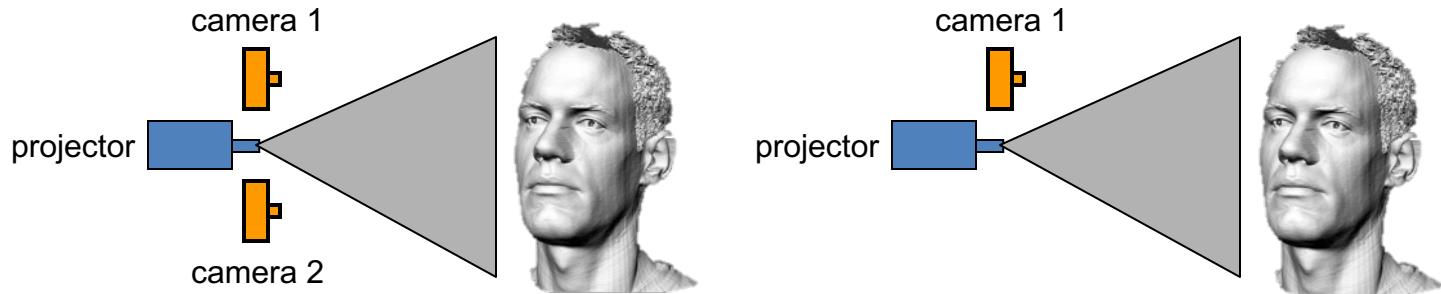
- Too small: large depth error
- Too large: difficult search problem

Stereo reconstruction pipeline

What will cause errors?

- Camera calibration errors
- Poor image resolution
- Occlusions
- Violations of brightness constancy (specular reflections)
- Textureless regions

Active stereo with structured light



- Project **“structured” infrared light patterns** onto the object
 - simplifies the correspondence problem
 - basis for active depth sensors, such as Kinect and iPhone X (using IR)

Active stereo with structured light



<https://ios.gadgethacks.com/news/watch-iphone-xs-30k-ir-dots-scan-your-face-0180944/>

Summary

- Passive stereo
- Active stereo
- Lidar (ToF)

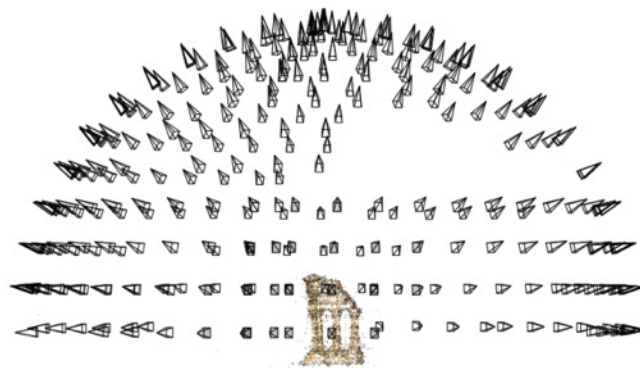


Questions?

Today

- Depth estimation
 1. Introduction
 2. Stereo matching
 - 3. Multi-view stereo**
- 3D reconstruction

Multi-view stereo



Multi-view stereo

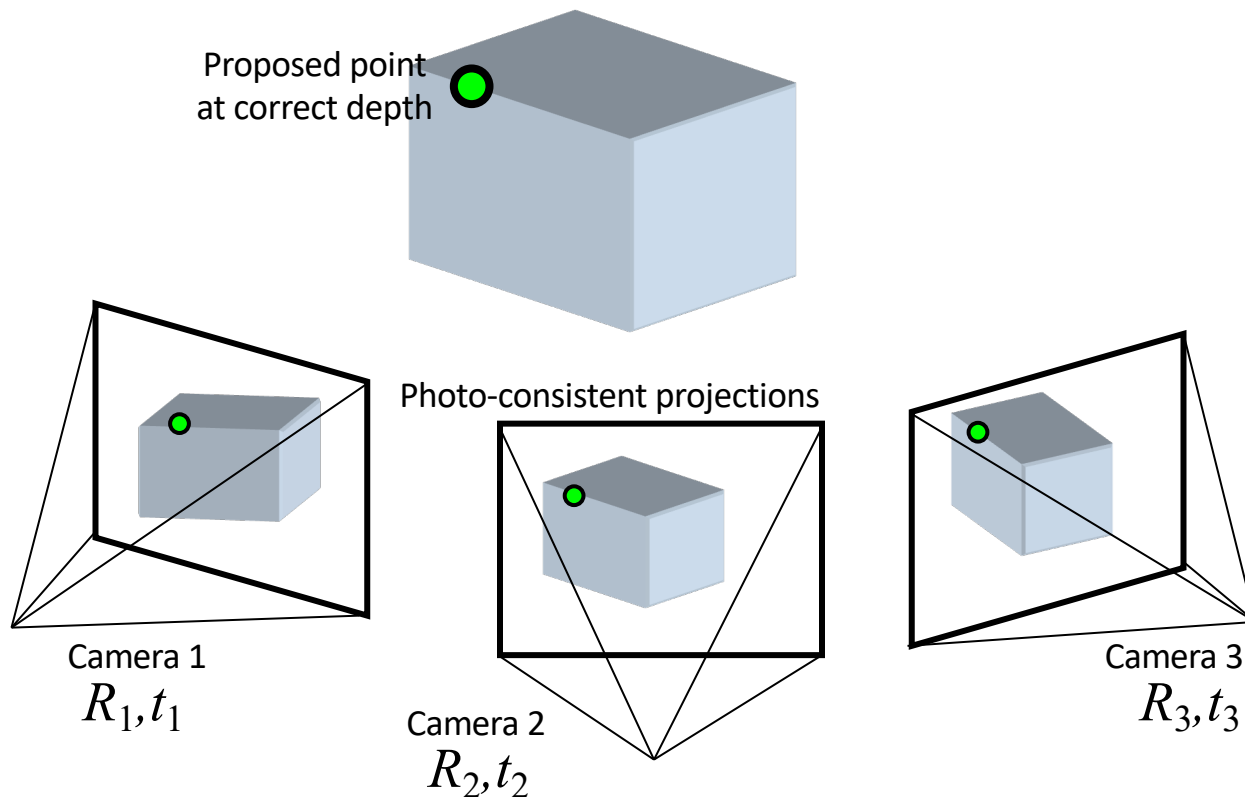


Multi-view stereo: advantages

- Can match windows using more than 1 neighbor, giving a **stronger constraint**
- If you have lots of potential neighbors, can **choose the best subset** of neighbors to match per reference image
- Can reconstruct a depth map for each reference frame, and then merge into a **complete 3D model**

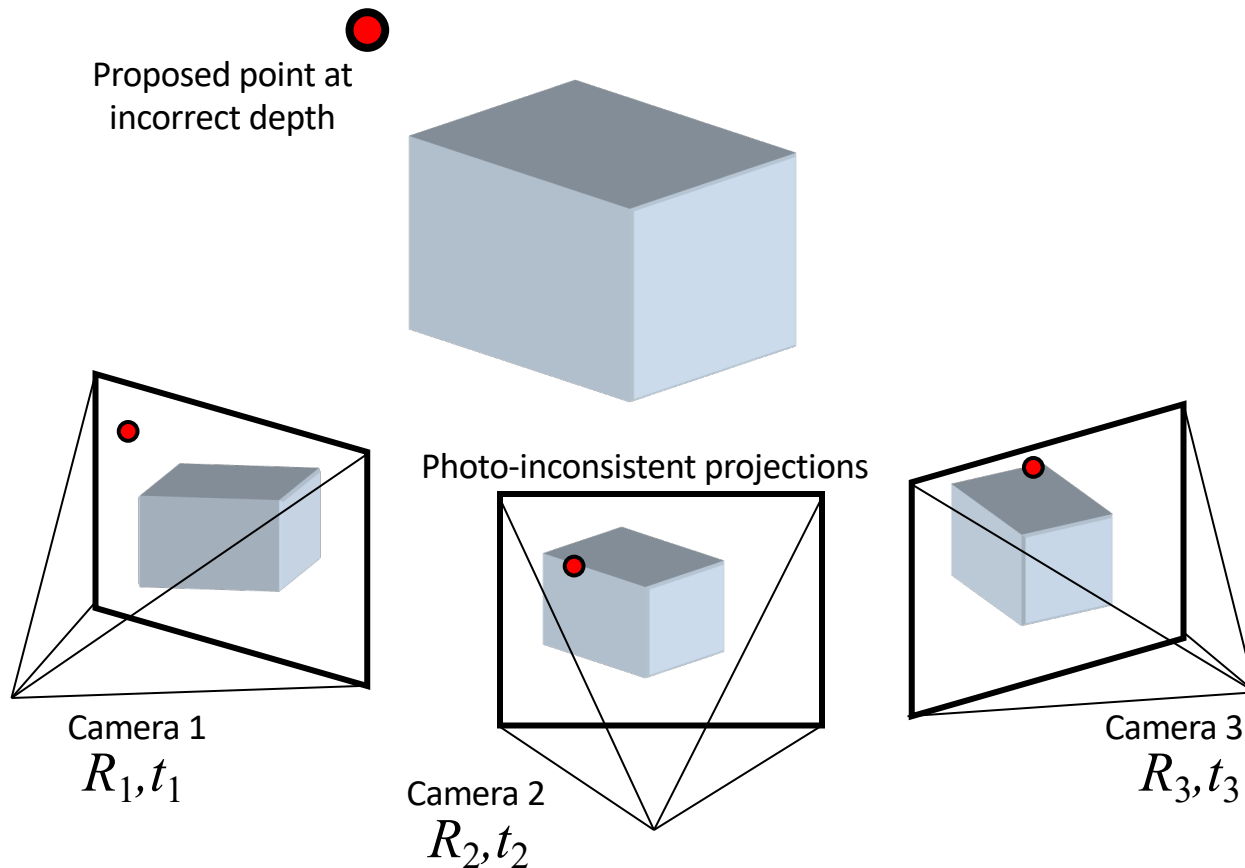
Multi-view stereo: basic idea

- Correct depth gives consistent projections



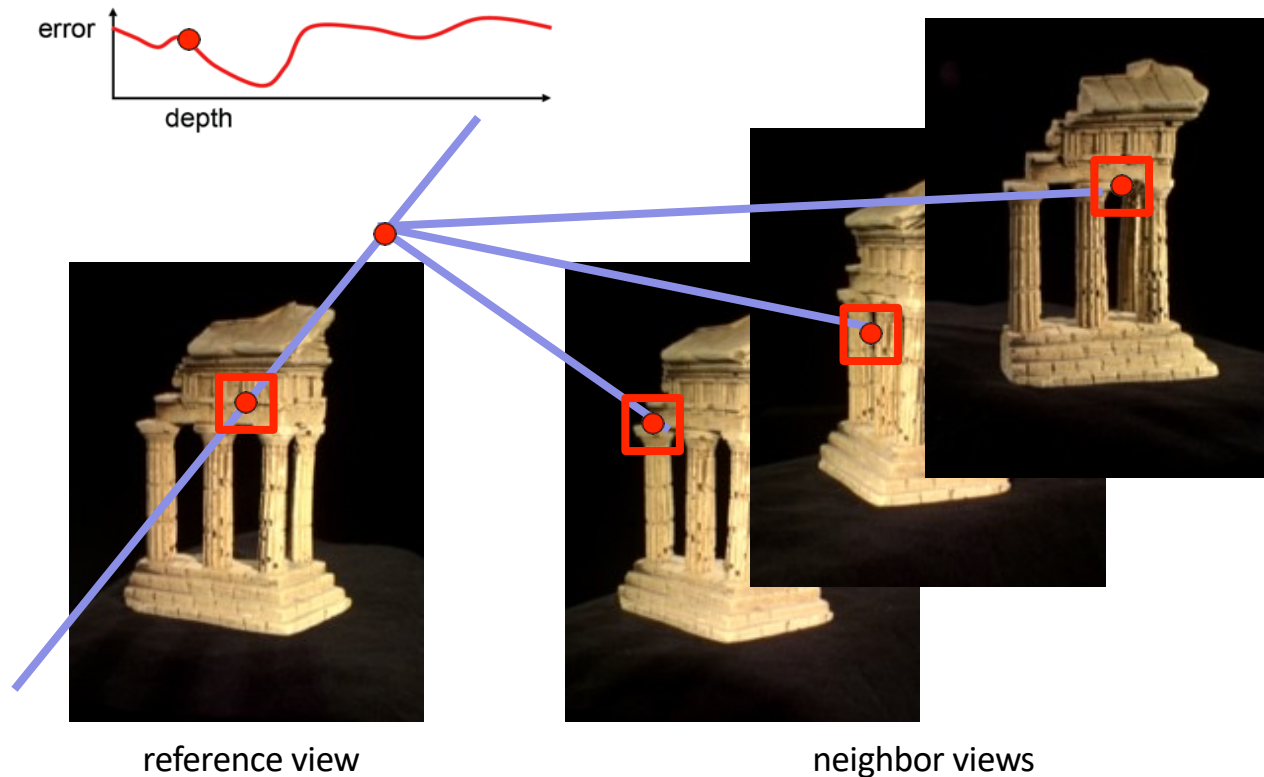
Multi-view stereo: basic idea

- Incorrect depth gives inconsistent projections



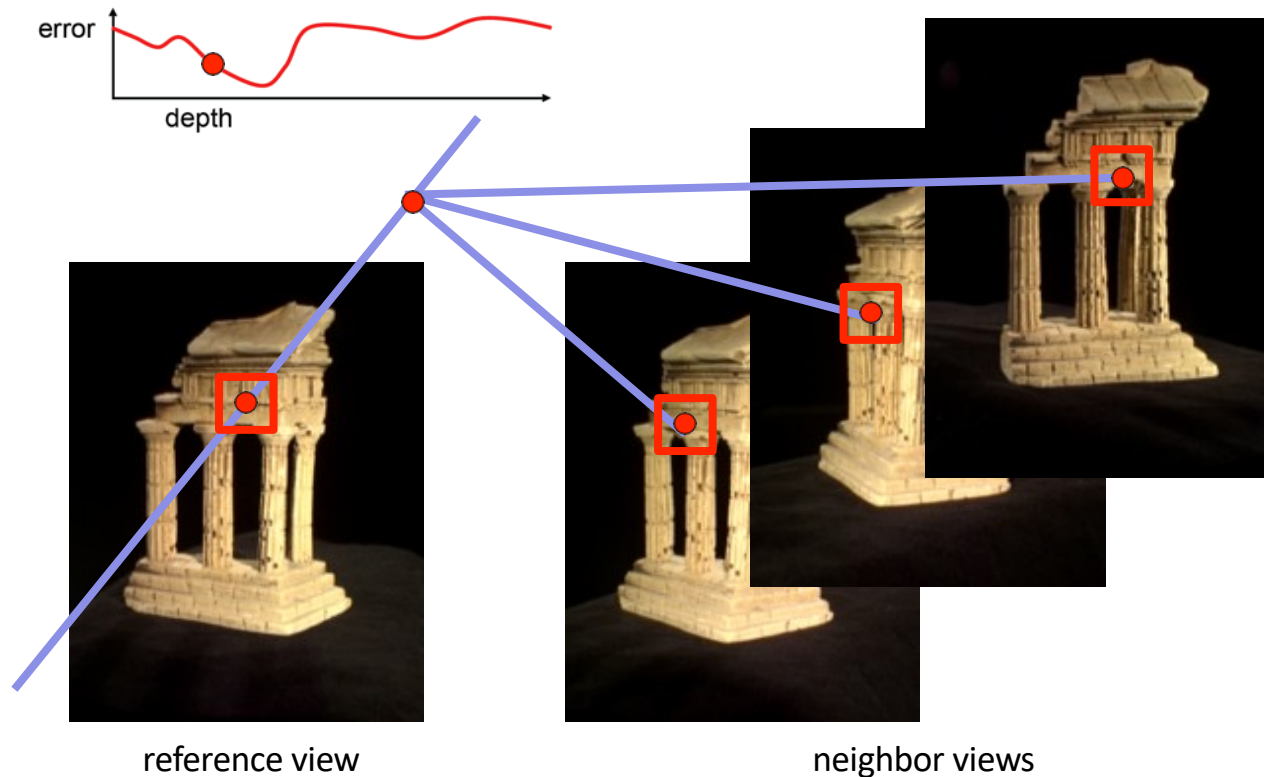
Multi-view stereo: basic idea

- Compute the error for each depth value for each point in the reference image



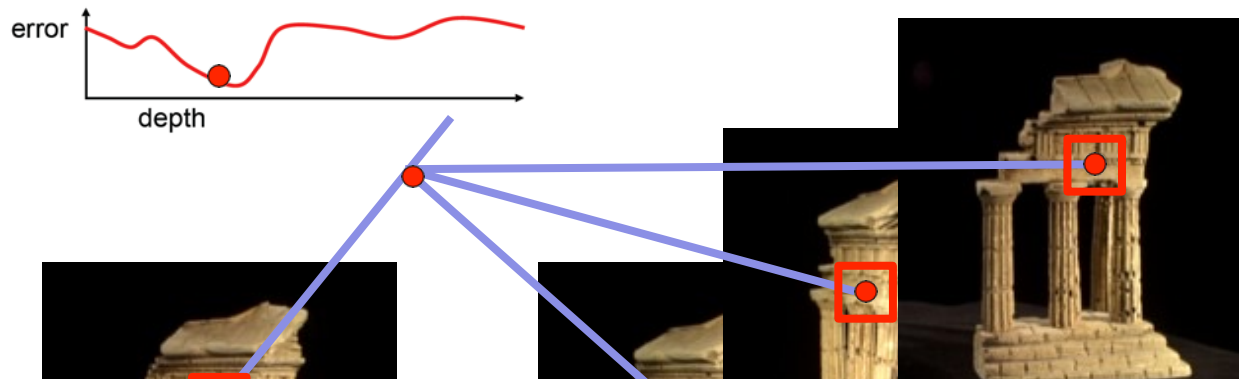
Multi-view stereo: basic idea

- Compute the error for each depth value for each point in the reference image



Multi-view stereo: basic idea

- Compute the error for each depth value for each point in the reference image



Find the depth value that gives the smallest error

reference view

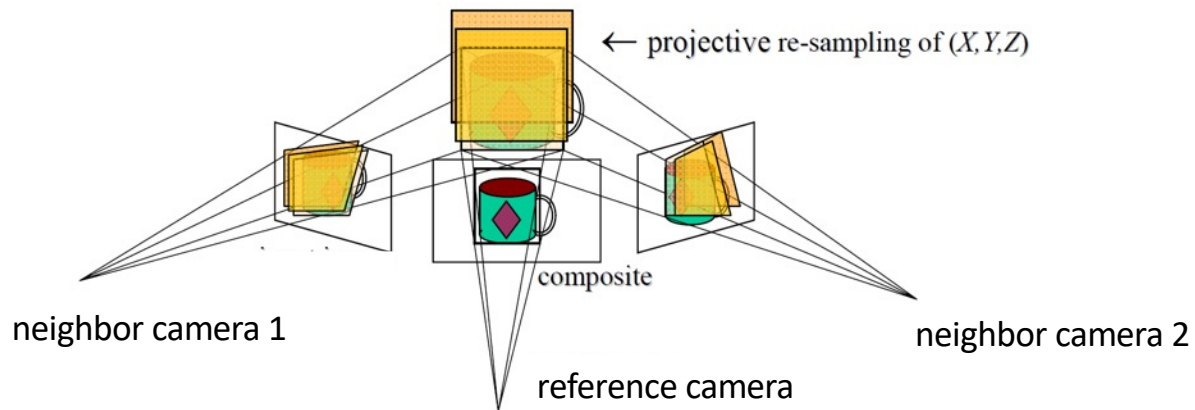
neighbor views

Multi-view stereo

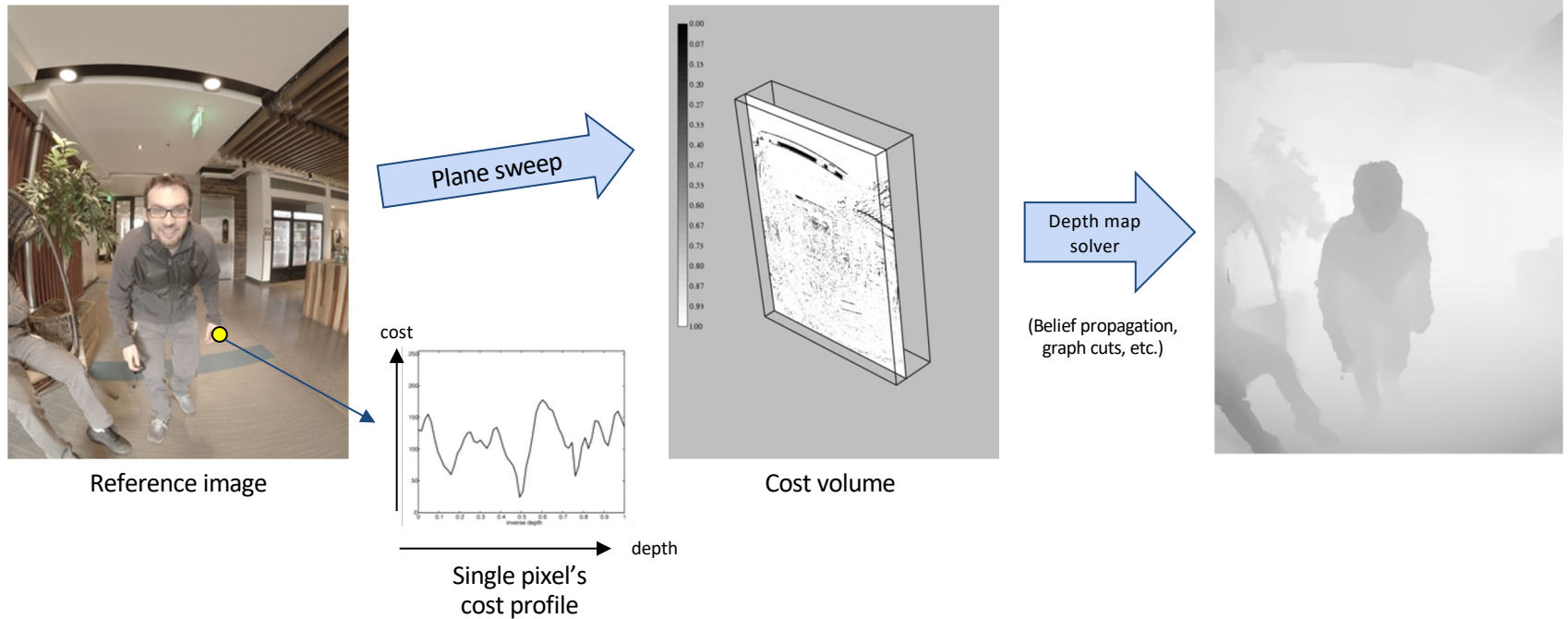
- How to compute the errors of all pixels at all depths efficiently?

Plane-Sweep

- Sweep family of planes parallel to the reference camera image plane
- Project each plane to neighbors views (via homography) and compare pixel values



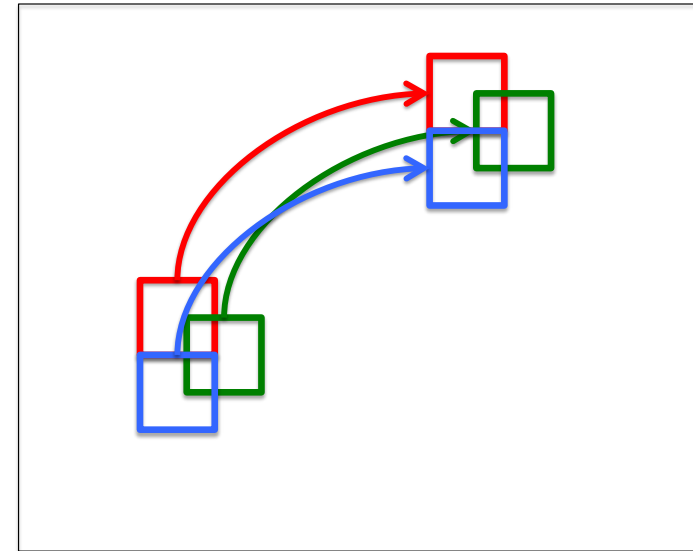
Cost Volumes -> Depth Maps



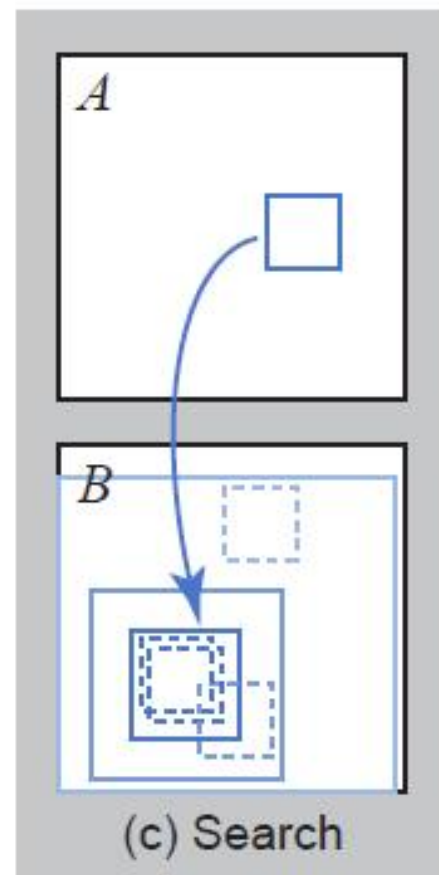
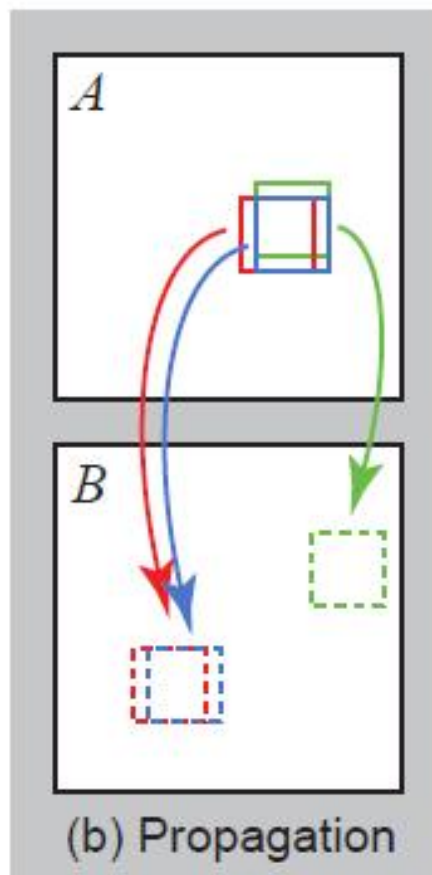
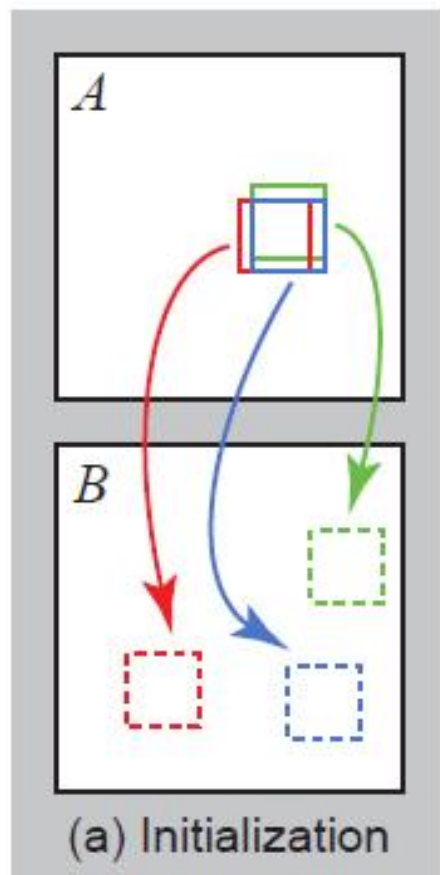
Cost volume is a 3D array that stores the errors of all pixels at all depths

PatchMatch

- An efficient algorithm for solving correspondence problems
- Assumptions
 - Large number of random sampling will yield some good guesses
 - Neighbors have similar offsets

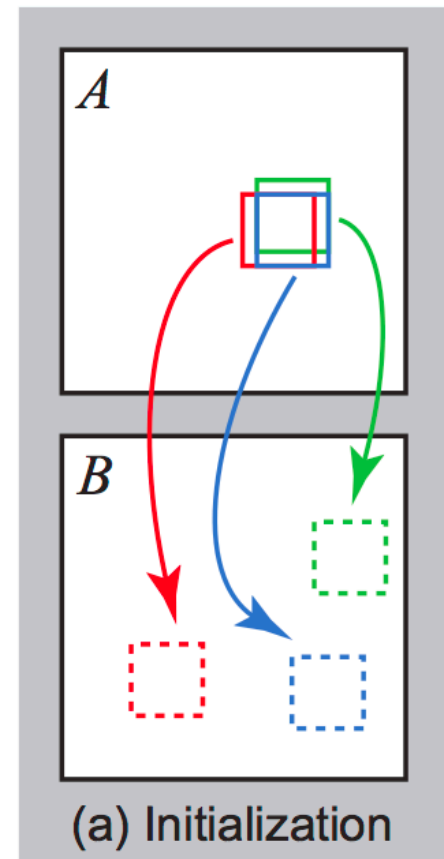


PatchMatch: 3 steps



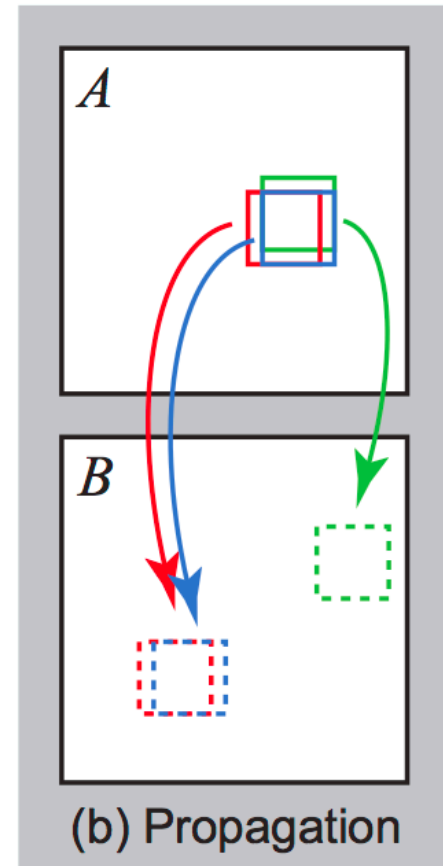
Step 1: random initialization

- Each pixel is given a random patch offset as initialization



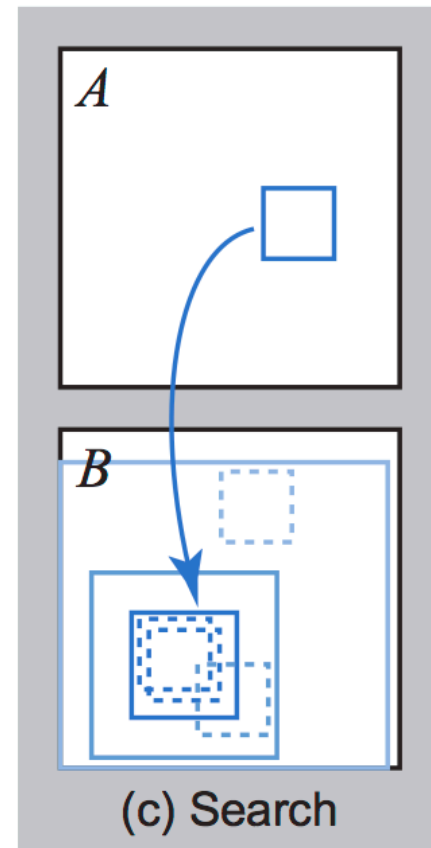
Step 2: propagation

- Each pixels checks if the offsets from neighboring patches give a better matching patch. If so, adopt neighbor's patch offset.



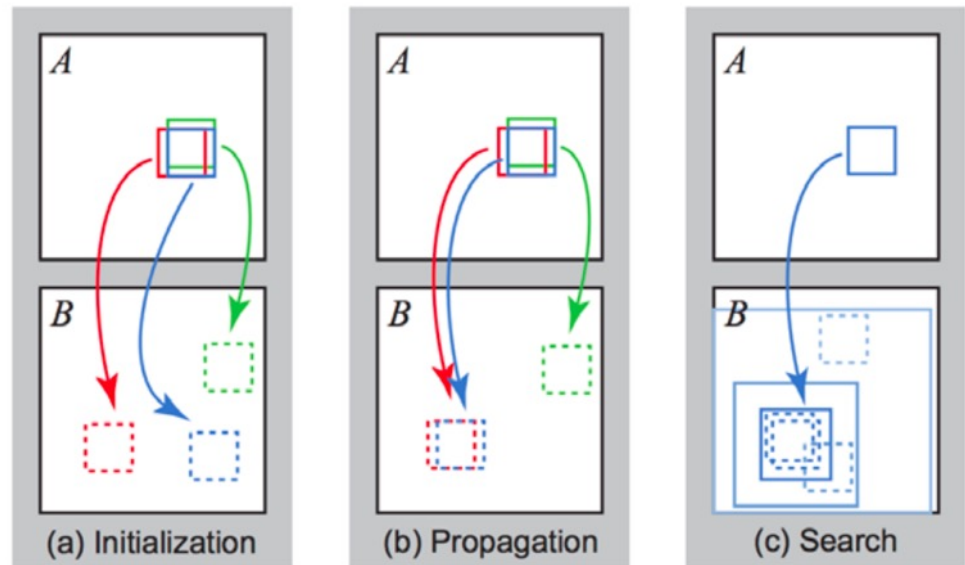
Step 3: local search

- Each pixels searches for better patch offsets within a concentric radius around the current offset.
- The search radius starts with the size of the image and is halved each time until it is 1.



PatchMatch: full algorithm

1. Initialize pixels with random patch offsets
2. Check if neighbors have better patch offsets
3. Search in concentric radius around the current offset for better patch offsets
4. Go to Step 2 until converge.



In MVS, replace patch offsets by depth values in the above algorithm

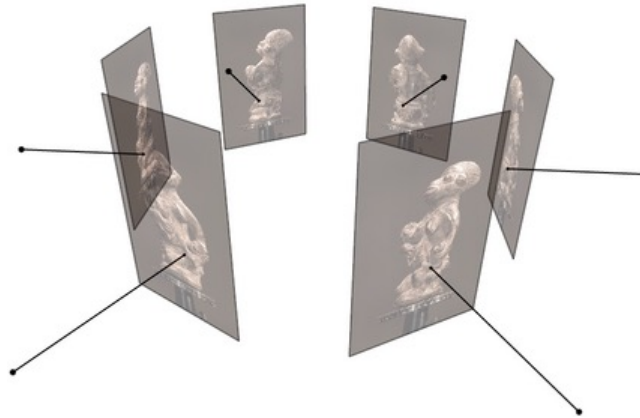
Questions?

Today

- Depth estimation
- **3D reconstruction**

3D reconstruction pipeline

- Compute depth map per image
- Fuse the depth maps into a 3D surface
- Texture mapping

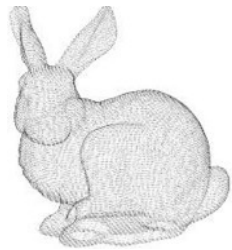


Figures by Carlos Hernandez

Today

- Depth estimation
- **3D reconstruction**
 1. **3D representations**
 2. 3D surface reconstruction
 3. Texture mapping

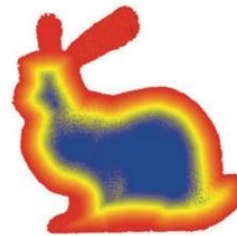
3D Representations



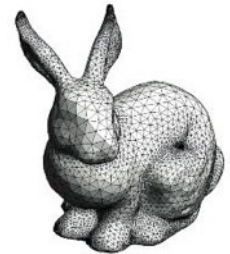
Point cloud



Occupancy



Signed distance



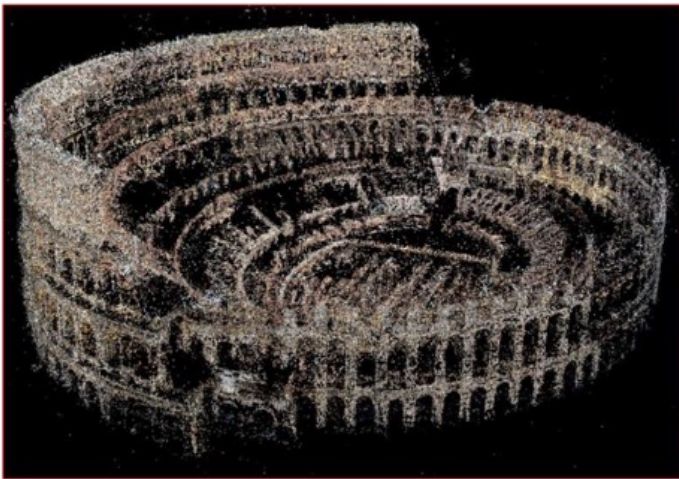
Mesh



Volume

3D Representations: point cloud

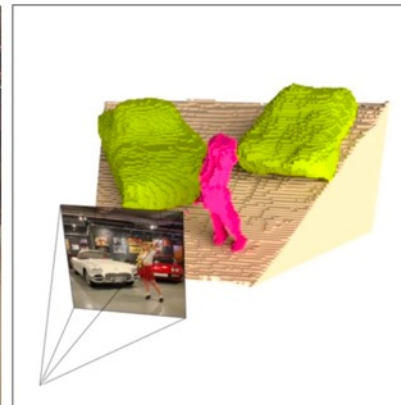
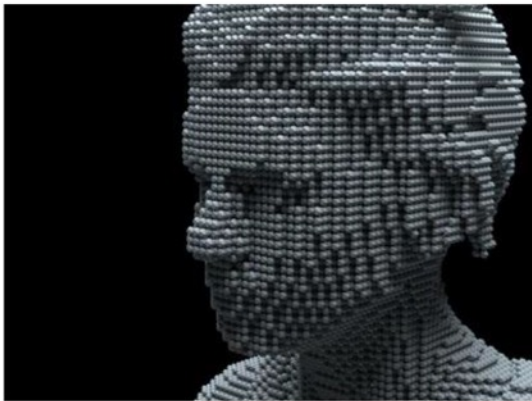
Point cloud: a set of 3D points



3D Representations: volume

Occupancy volume: a 3D volume with each voxel being

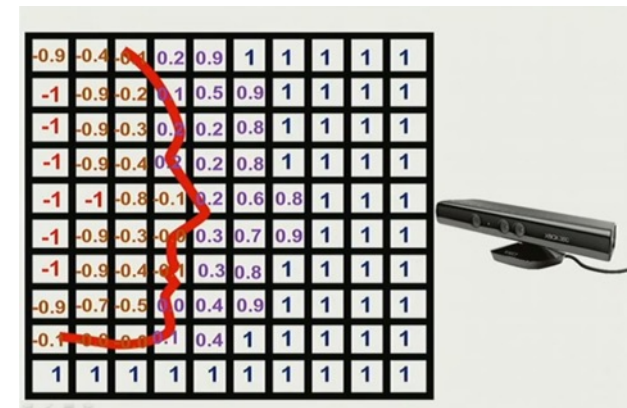
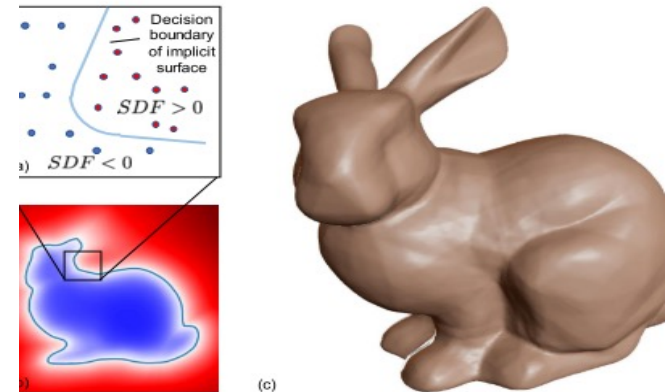
$$V_{ijk} = \begin{cases} 1 & \text{if occupied} \\ 0 & \text{if empty} \end{cases}$$



3D Representations: volume

SDF volume

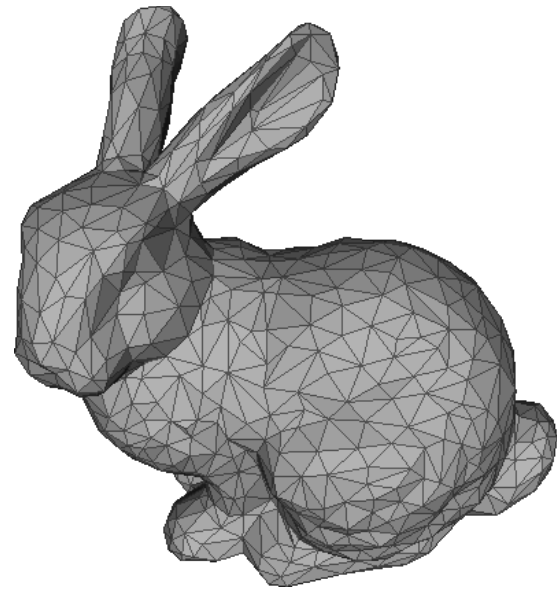
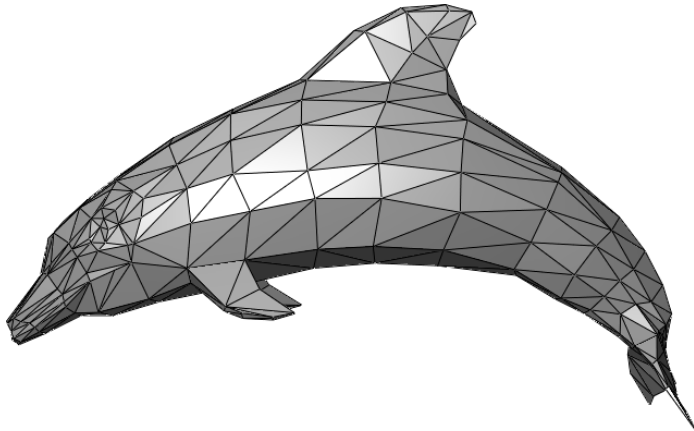
- Signed Distance Function (SDF)
 - The **distance** of a point to the **shape boundary**
 - The distance is defined by a metric, usually the Euclidean distance
- Truncated Signed Distance Function (**TSDF**)
 - Truncation SDF's distance value to $[-1, 1]$



3D Representation: mesh

A polygon mesh with vertices and edges

- Usually triangle mesh



Today

- Depth estimation
- 3D reconstruction
 1. 3D representations
 2. 3D surface reconstruction
 3. Texture mapping

3D surface reconstruction

How to fuse depth maps into a complete 3D mesh?

1. Depth maps -> Occupancy volume
 - Poisson reconstruction
2. Occupancy volume -> mesh
 - Marching cubes

Why first constructing a volume?

- Easier to be converted to mesh
- Good for denosing

Poisson reconstruction

Convert **depth maps** into a **3D volume**?

Poisson Surface Reconstruction

Michael Kazhdan¹, Matthew Bolitho¹ and Hugues Hoppe²

¹Johns Hopkins University, Baltimore MD, USA

²Microsoft Research, Redmond WA, USA

Poisson reconstruction

- Convert depth map to point cloud



$$\begin{cases} z = \text{depth}(i, j) \\ x = \frac{(j - c_x) \times z}{f_x} \\ y = \frac{(i - c_y) \times z}{f_y} \end{cases}$$

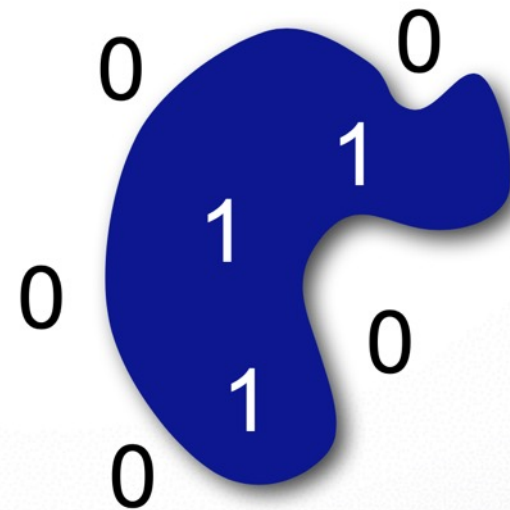
- Compute normal vector for each point



Poisson reconstruction

Represent surface by indicator (occupancy) function

$$\chi_M(p) = \begin{cases} 1 & \text{if } p \in M \\ 0 & \text{if } p \notin M \end{cases}$$

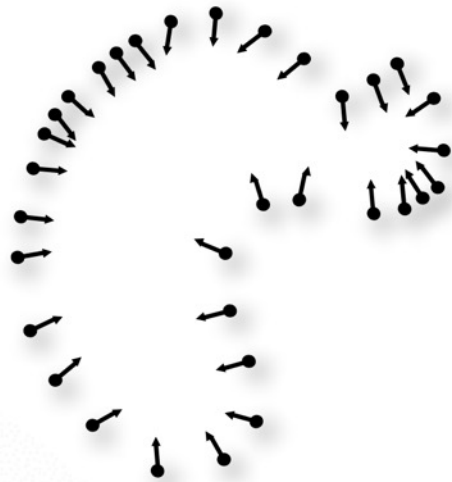


Indicator function

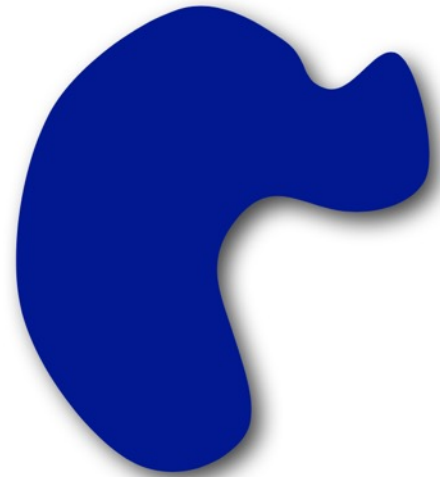
χ_M

Poisson reconstruction

How to construct the indicator function?



Oriented points



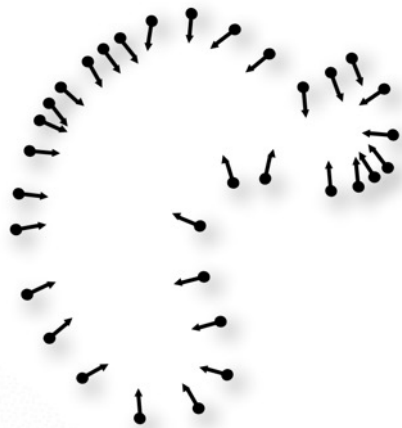
Indicator function

$$\chi_M$$

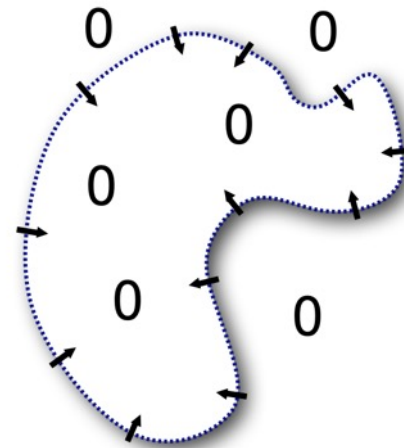
Poisson reconstruction

Gradient Relationship

- There is a relationship between the **normals** of the points and **gradient** of indicator function



Oriented points



Indicator gradient

$$\nabla \chi_M$$

Poisson reconstruction

- Represent the oriented points by a **vector field** $\vec{V}$
- Find the function χ whose gradient best **approximates** $\vec{V}$ by minimizing:

$$\min_{\chi} \|\nabla \chi - \vec{V}\|$$

- Solved by Poisson equation

Marching cubes

- Till now we can get volumetric representation from depth maps
- How to extract **3D surface (mesh)** from **volumetric representation**?

MARCHING CUBES: A HIGH RESOLUTION 3D SURFACE CONSTRUCTION ALGORITHM

*William E. Lorensen
Harvey E. Cline*

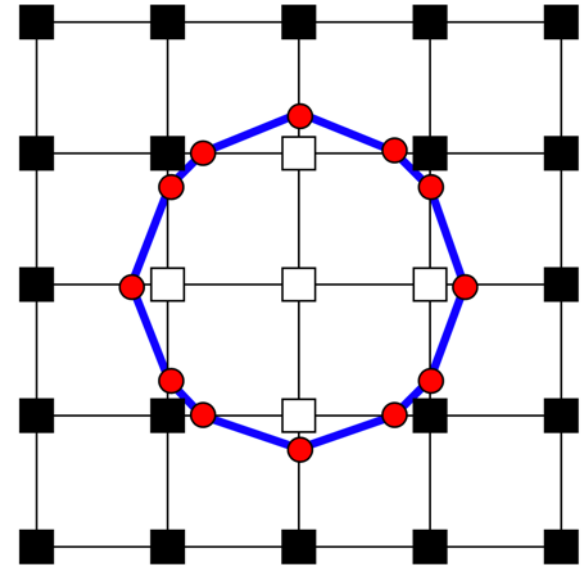
General Electric Company
Corporate Research and Development
Schenectady, New York 12301

"Marching Cubes: A High Resolution 3D Surface Construction Algorithm",
William E. Lorensen and Harvey E. Cline,
Computer Graphics (Proceedings of SIGGRAPH '87).

Marching Squares (2D)

For each grid cell with a **sign change**

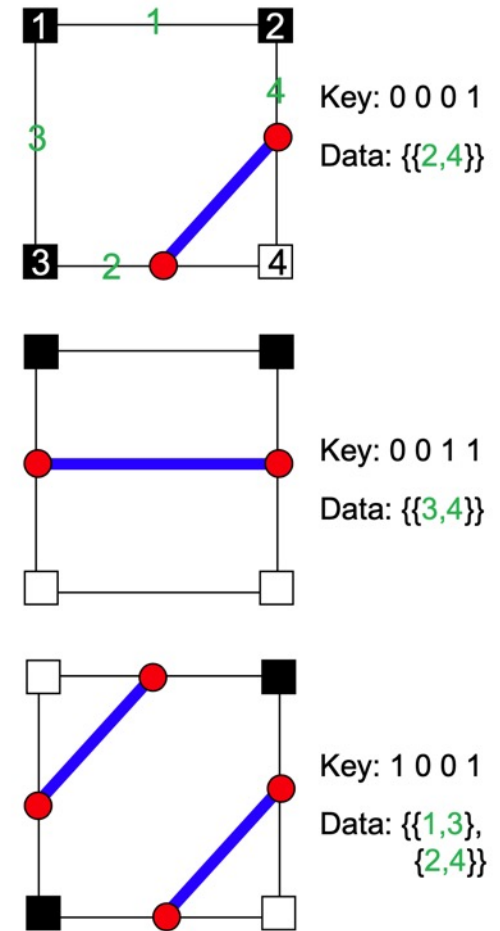
- Create one vertex on each grid edge with a sign change.
- Connect vertices by lines.
- Location of the vertex can be determined by linear interpolation of SDF values.



Marching Squares (2D)

Connect vertices by lines

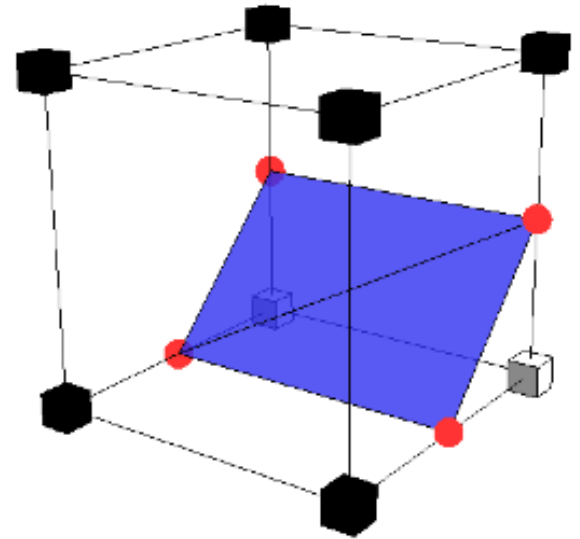
- Lines should **not intersect**
- Use a pre-computed **look-up table**



Marching Cubes (3D)

For each grid cell with a sign change

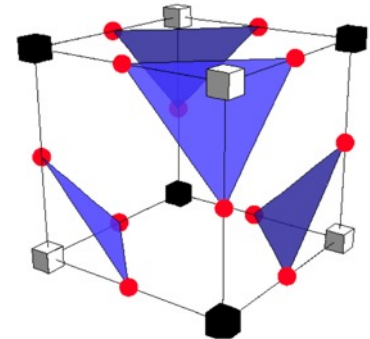
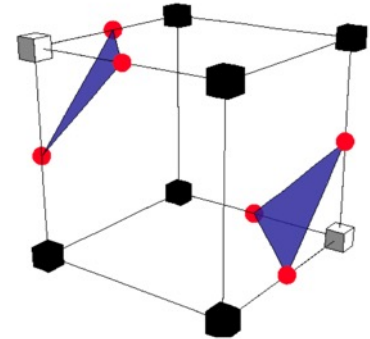
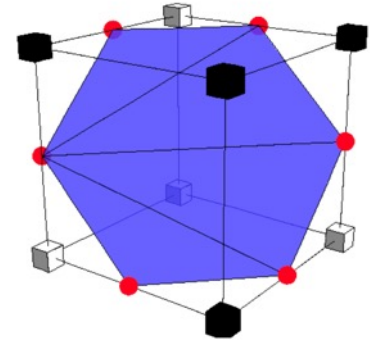
- Create one vertex on each grid edge with a sign change
- Connect vertices by **triangles**



Marching Cubes (3D)

Connect vertices by **triangles**

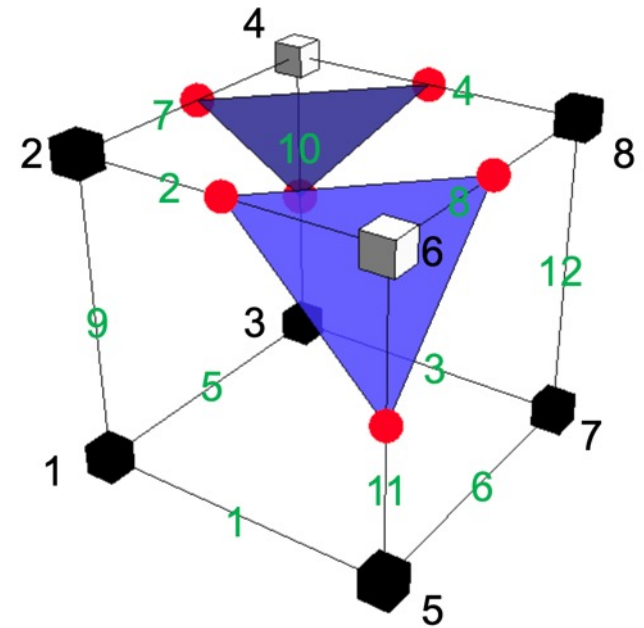
- Triangles should **not intersect**
- Much more complicated than 2D cases



Marching Cubes (3D)

Look-up table

- $2^8=256$ sign configurations
- For each sign configuration, it stores indices of the grid edges whose vertices make up the triangles



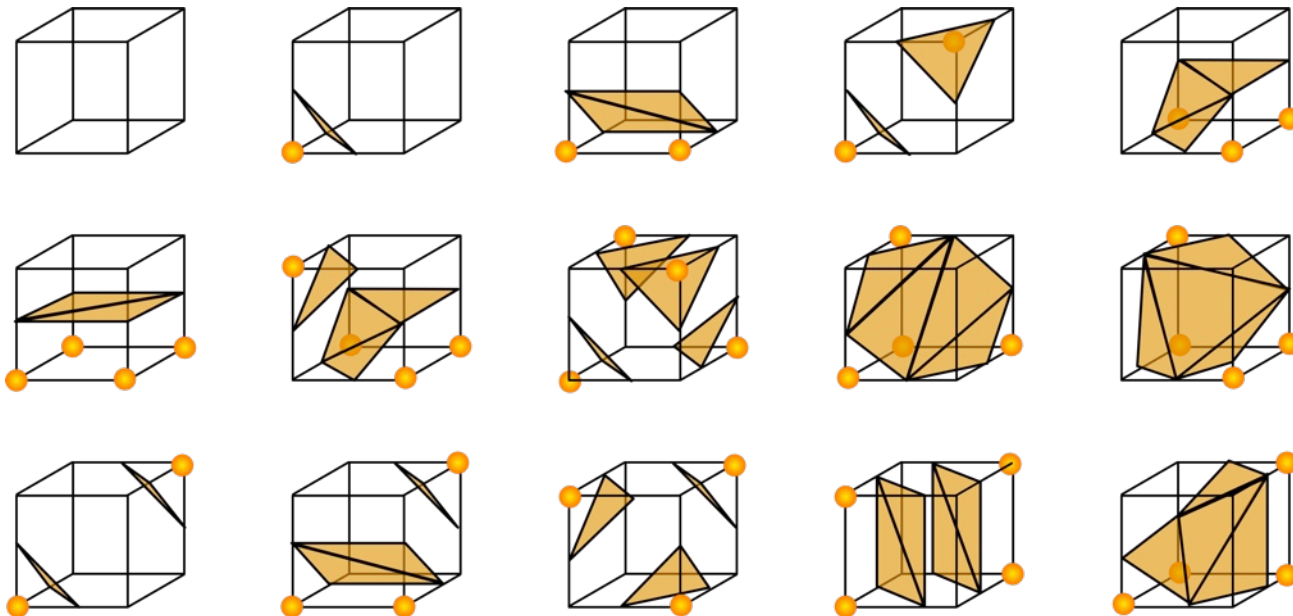
Sign: "0 0 0 1 0 1 0 0"

Triangles: $\{\{2, 8, 11\}, \{4, 7, 10\}\}$

Marching Cubes (3D)

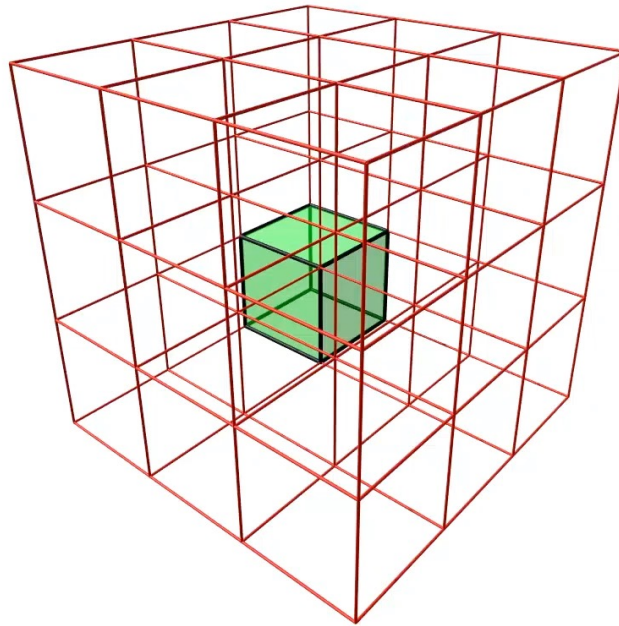
Look-up table

- $2^8=256$ sign configurations
- 15 configurations in total (considering symmetry)

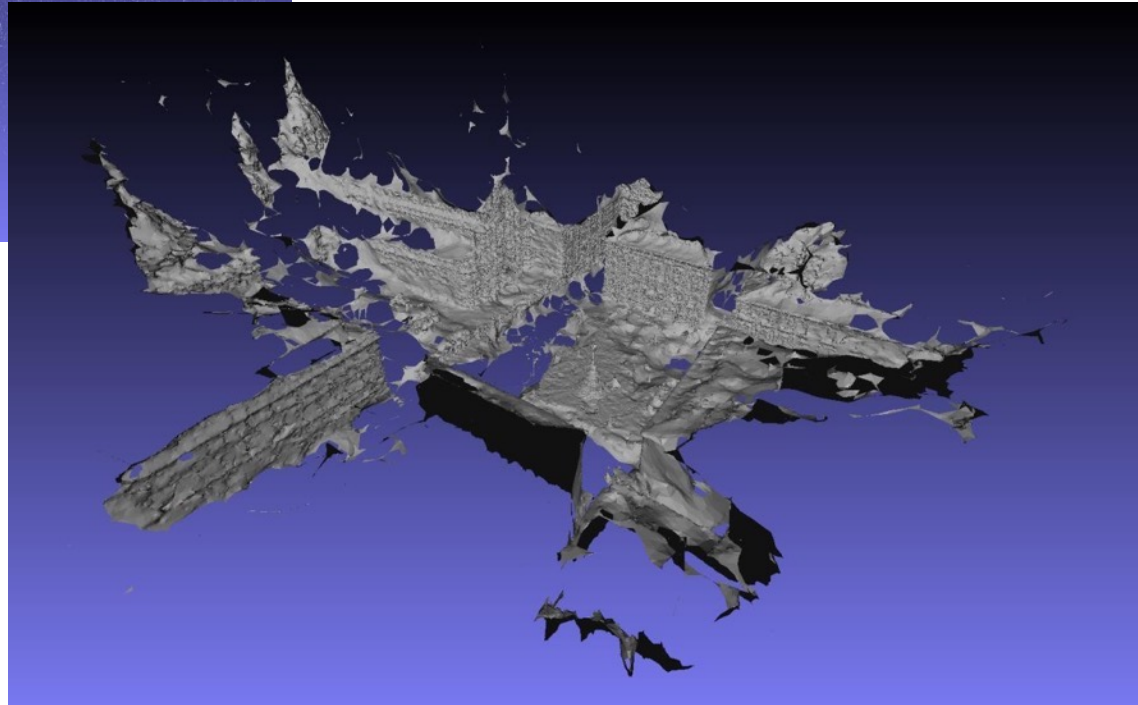
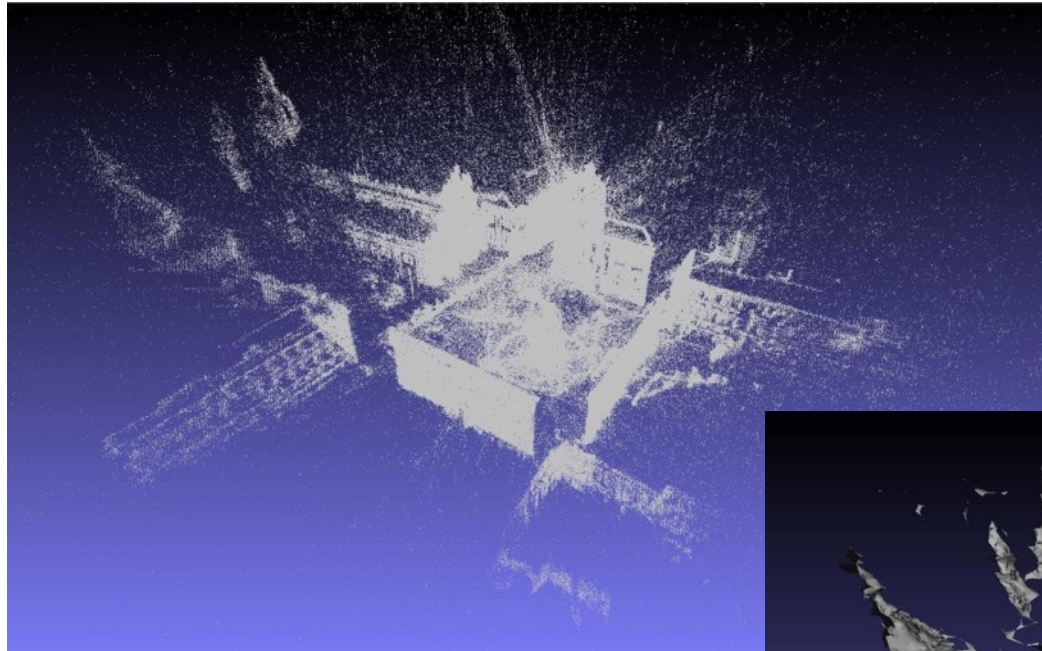


Marching Cubes (3D)

Marching cubes



Surface Extraction Results



COLMAP



Dense models of several landmarks produced by COLMAP's MVS pipeline.

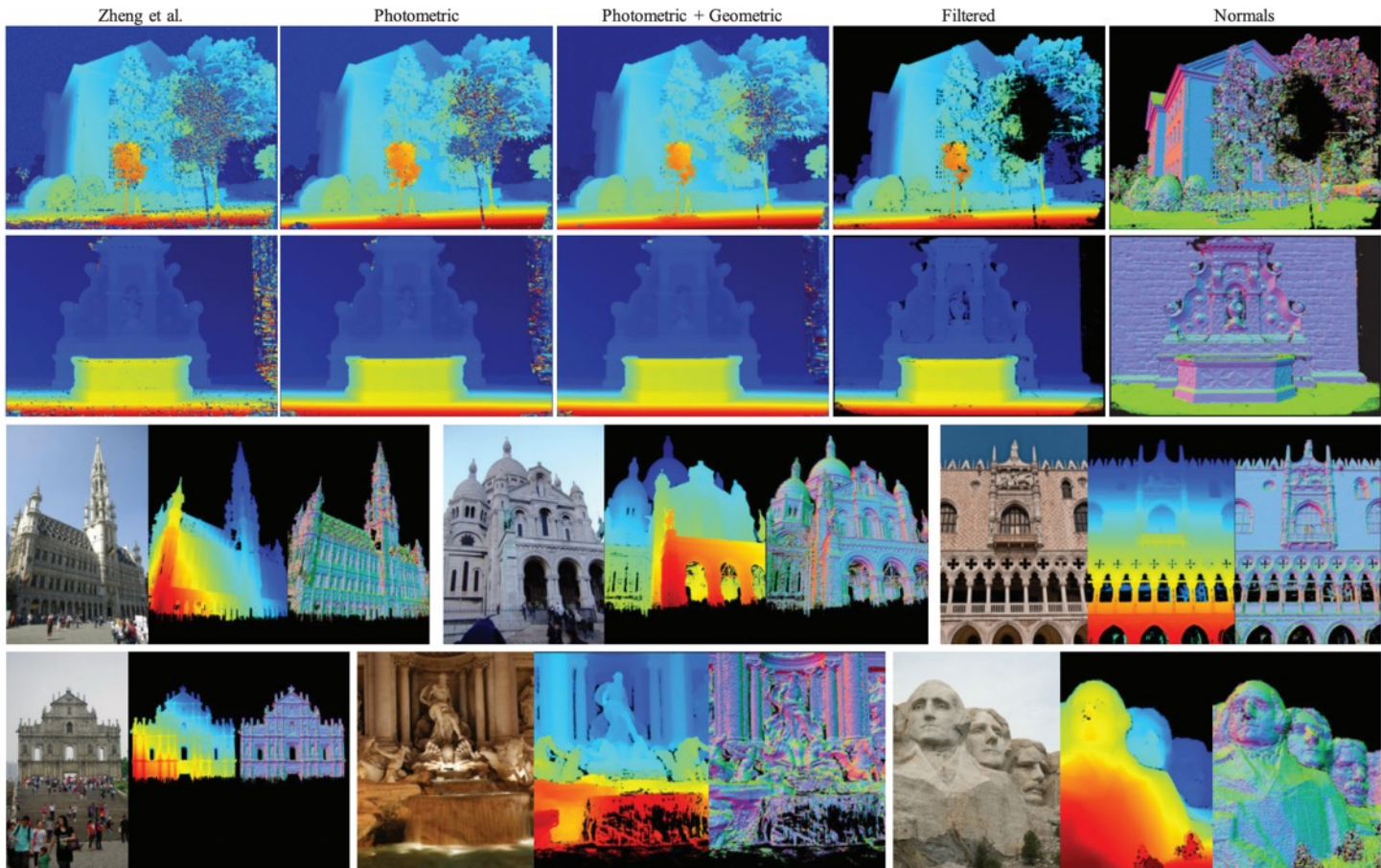
COLMAP is a general-purpose **Structure-from-Motion (SfM)** and **Multi-View Stereo (MVS)** pipeline with a graphical and command-line interface. It offers a wide range of features for reconstruction of **ordered and unordered image collections**.

Homepage: colmap.github.io

Paper: Pixelwise view selection for unstructured multi-view stereo, Schonberger J L, Zheng, ECCV 2016.

MVS in COLMAP

Results

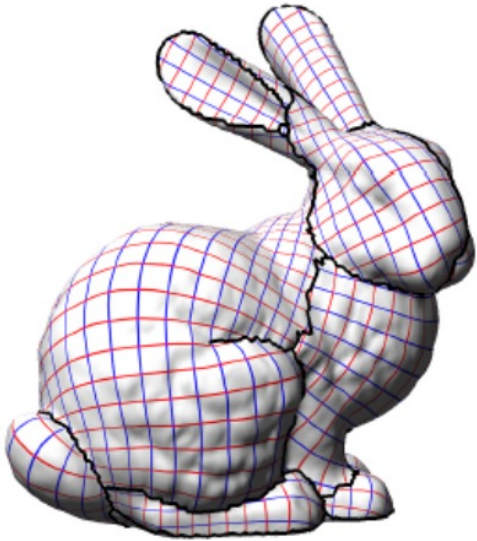


Today

- Depth estimation
- 3D reconstruction
 1. 3D representations
 2. 3D surface reconstruction
 3. Texture mapping

Texture Mapping

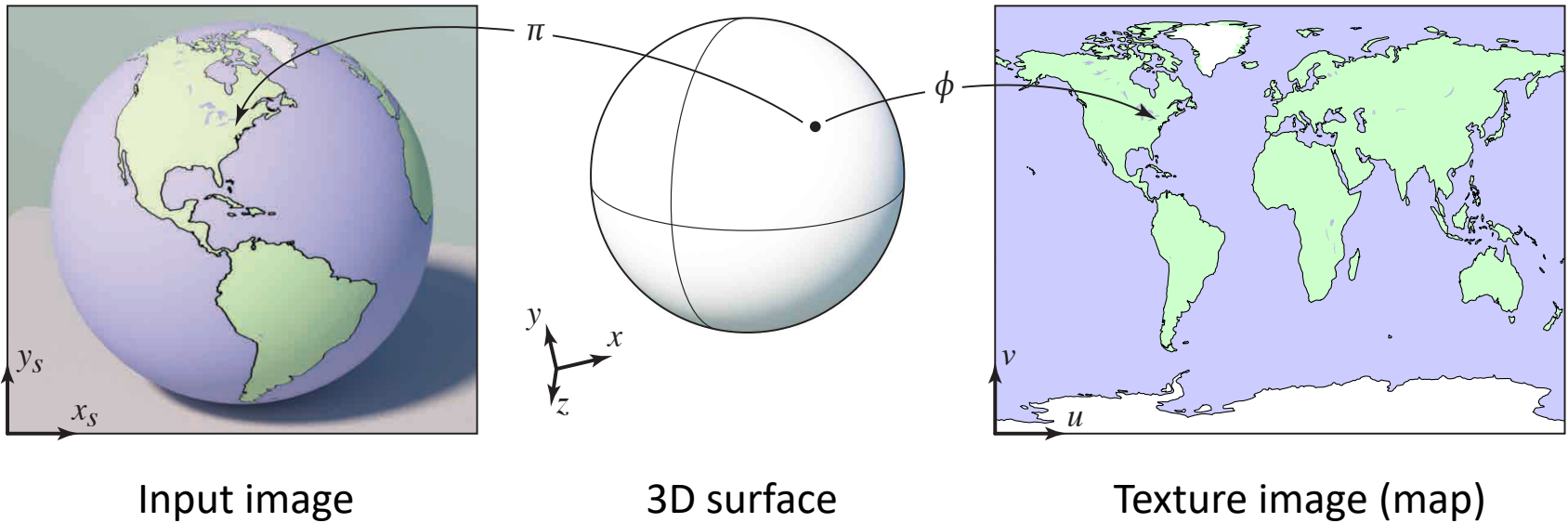
Texture mapping: “add color” to 3D surface.



Texture Mapping

Surface lives in 3D world space

Every 3D surface point also has a place where it goes in the 2D image (**texture**).

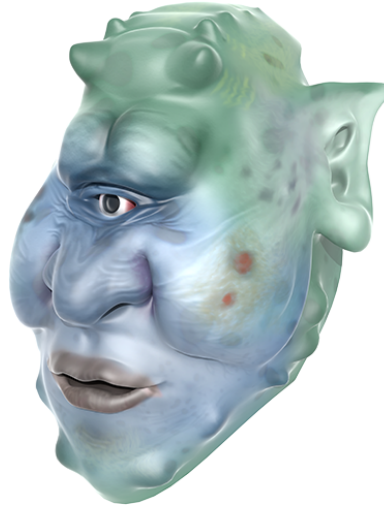


Texture Mapping

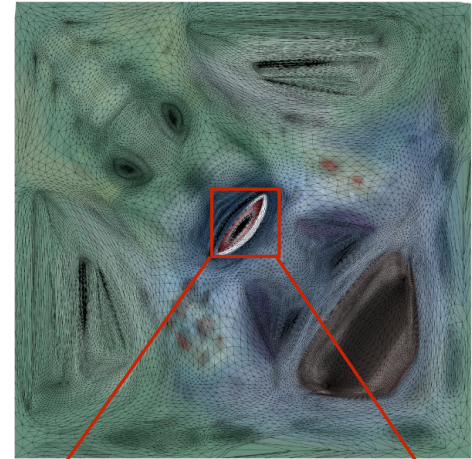
Rendering without texture



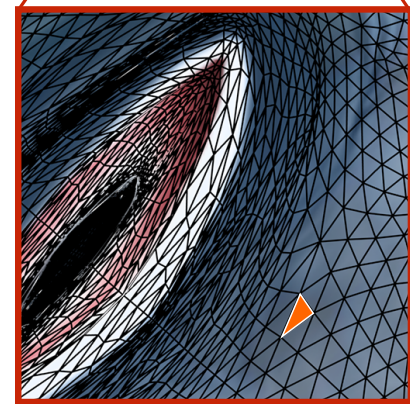
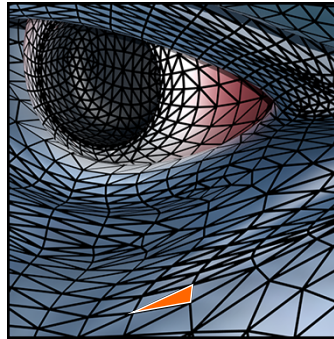
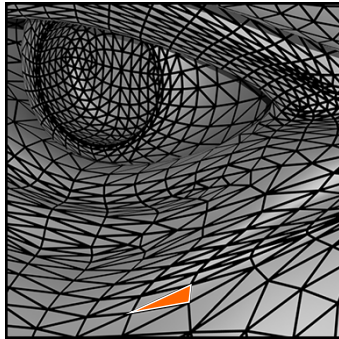
Rendering with texture



Texture



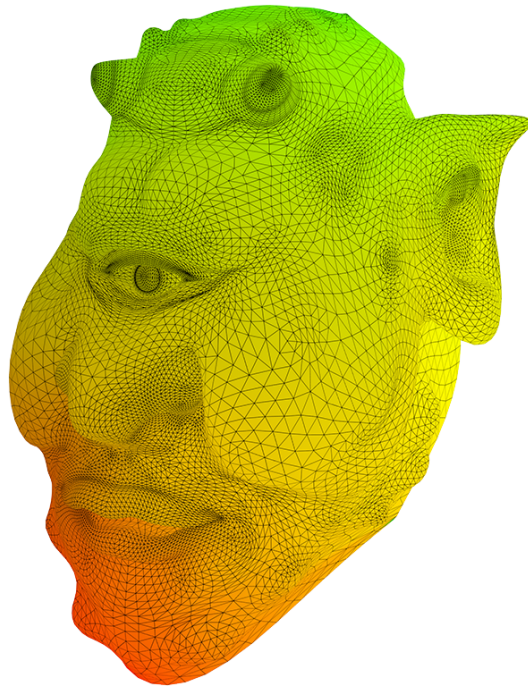
Zoom



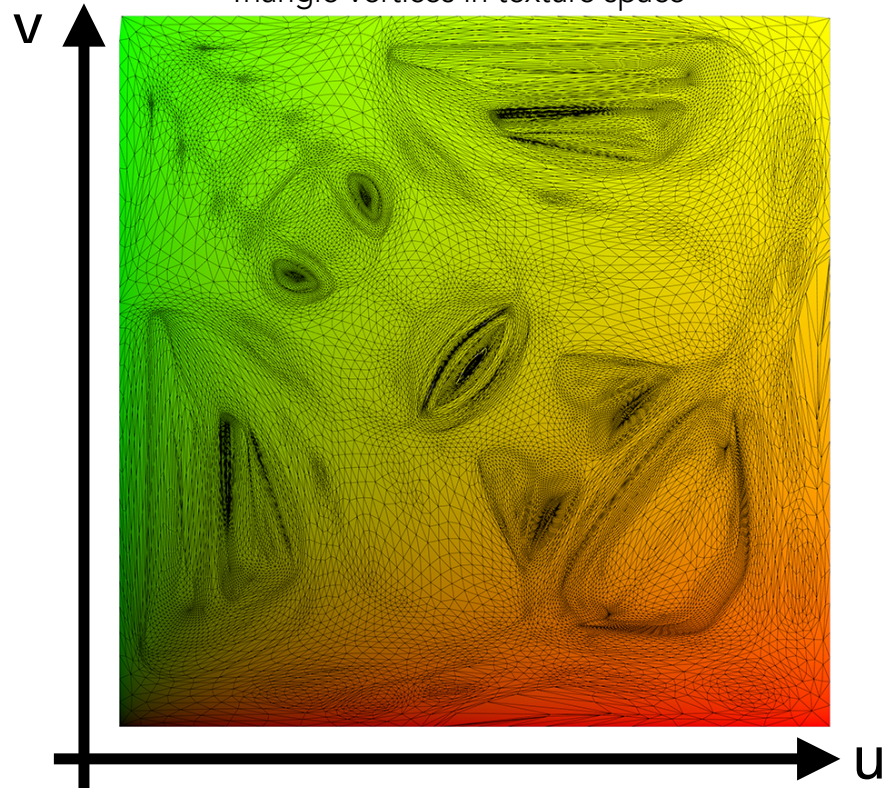
Each triangle "copies" a piece of the texture image to the surface.

Texture Mapping

Visualization of texture coordinates



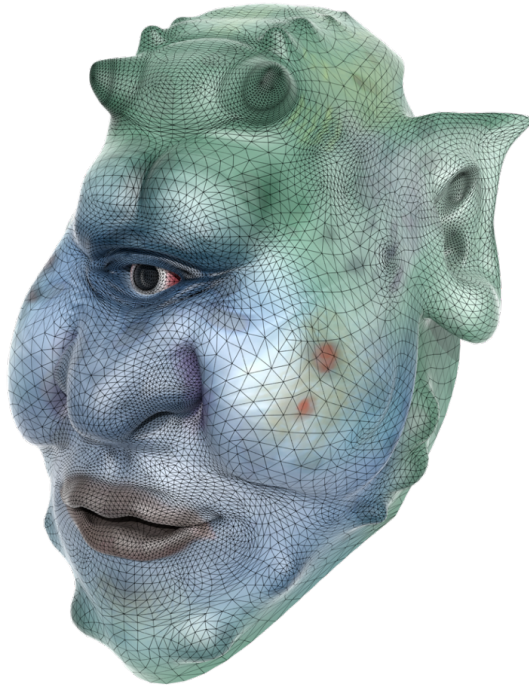
Triangle vertices in texture space



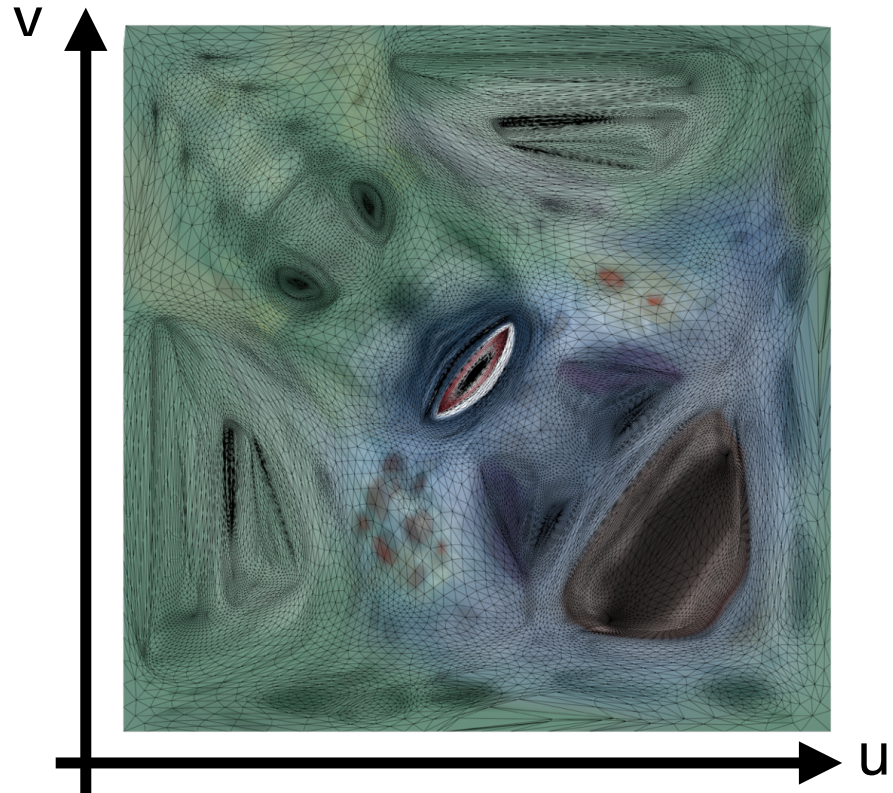
Each triangle vertices is assigned a 2D coordinate (u,v) in the texture image

Texture Mapping

Rendered result



Triangle vertices in texture space

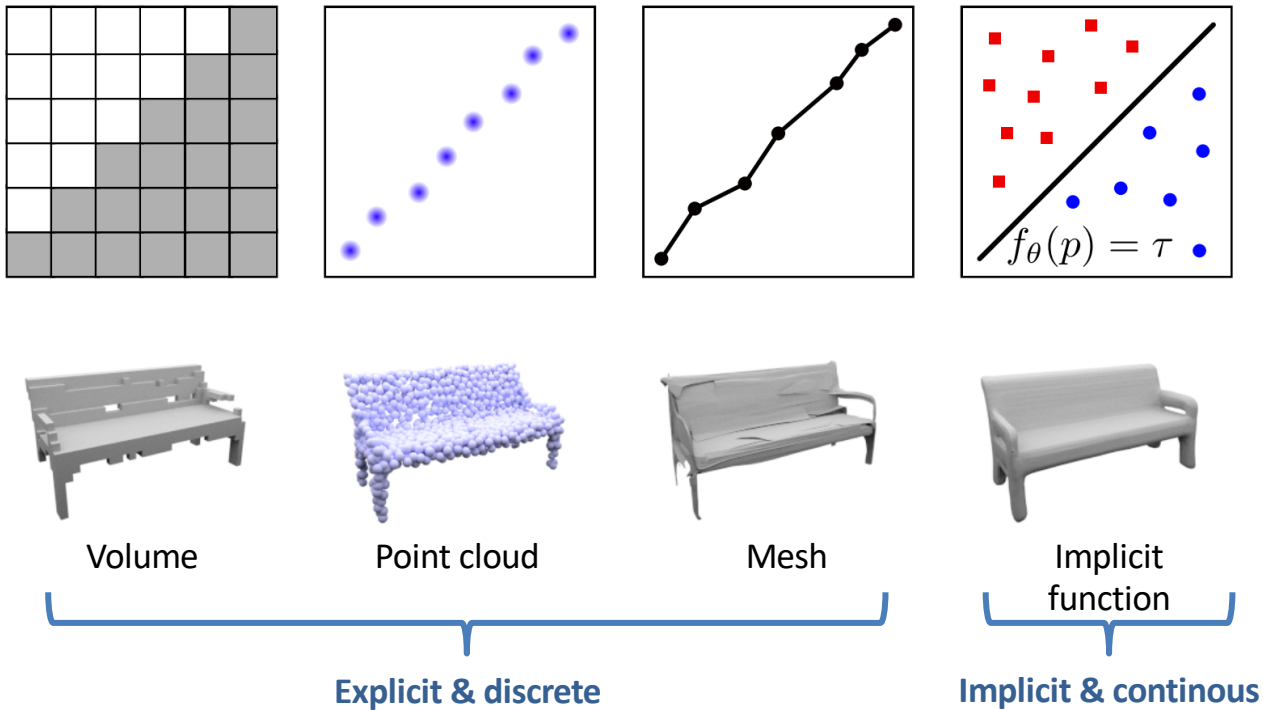


Each triangle vertices is assigned a 2D coordinate (u,v) in the texture image

Today

- Depth estimation
- 3D reconstruction
 1. 3D representations
 2. 3D surface reconstruction
 3. Texture mapping
- **Neural Scene Representations**

Implicit Representations

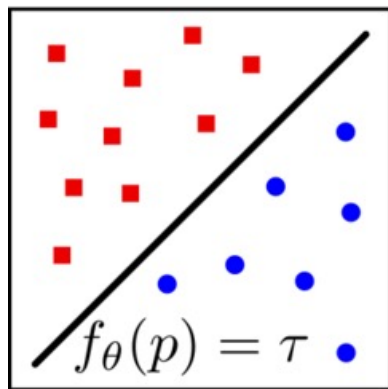


Occupancy Networks: Learning 3D Reconstruction in Function Space,
CVPR 2019

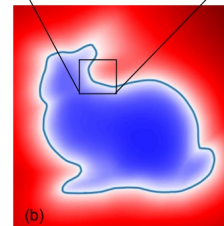
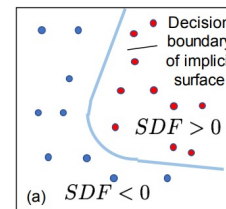
Implicit Representations

The implicit function can be:

Occupancy



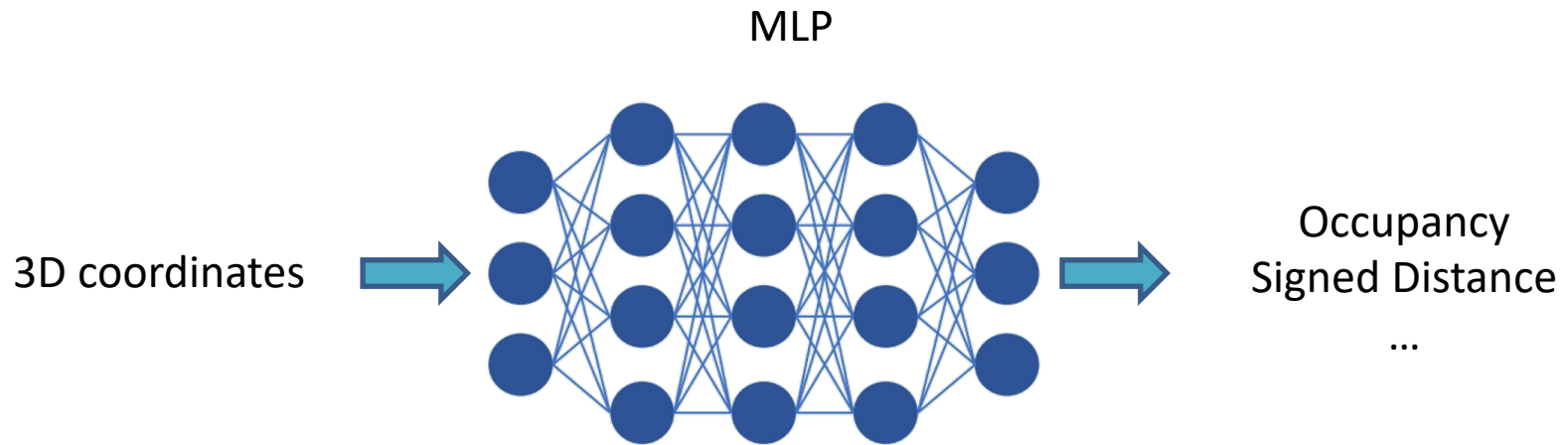
Signed distance function (SDF)



(c)



Implicit Neural Representations



Occupancy Networks: Learning 3D Reconstruction in Function Space, *CVPR* 2019.

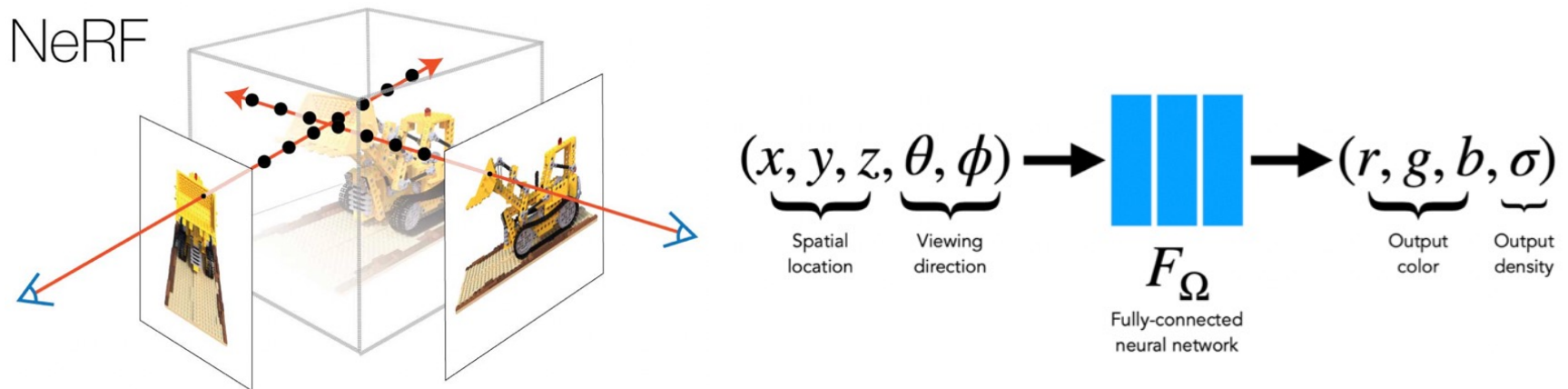
DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation, *CVPR* 2019.

Learning implicit fields for generative shape modeling, *CVPR* 2019.

Scene Representation Networks: Continuous 3D-Structure-Aware Neural Scene Representations, *NeurIPS* 2019.

Neural Radiance Fields (NeRF)

Representing scenes as **continuous density and color fields**



NeRF: Representing scenes as neural radiance fields for view synthesis,
ECCV 2020.

Neural Radiance Fields (NeRF)

Volume rendering, which is **differentiable**

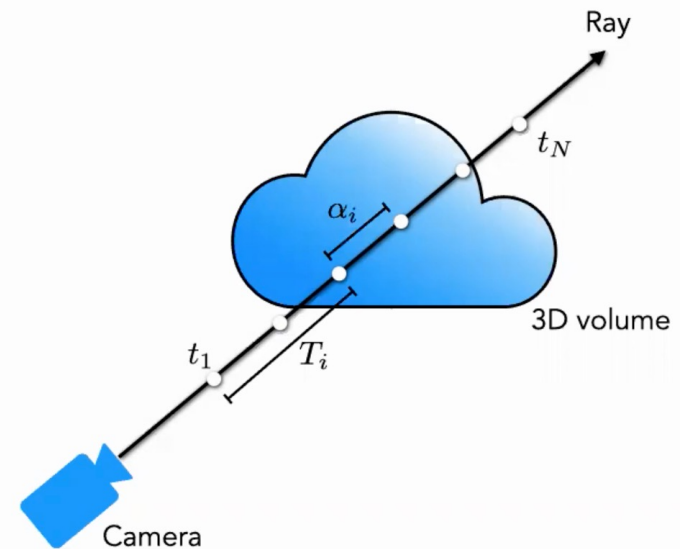
Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors

How much light is blocked earlier along ray:

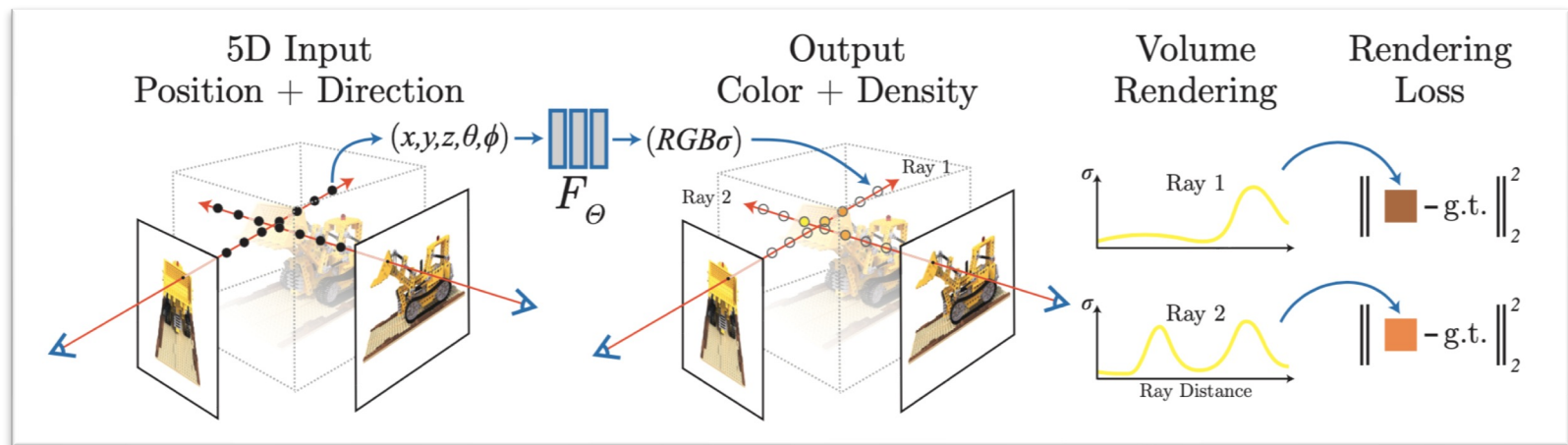
$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$



NeRF: Representing scenes as neural radiance fields for view synthesis,
ECCV 2020.

Neural Radiance Fields (NeRF)

Reconstruct NeRF from images



NeRF: Representing scenes as neural radiance fields for view synthesis, *ECCV 2020*.

Neural Radiance Fields (NeRF)



NeRF: Representing scenes as neural radiance fields for view synthesis, *ECCV 2020*.

Neural Radiance Fields (NeRF)

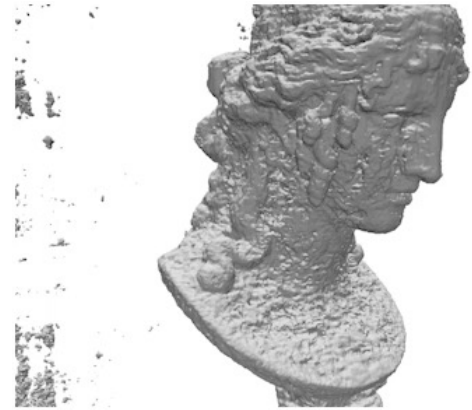
Limitations: Poor surface reconstruction quality



Reference
image



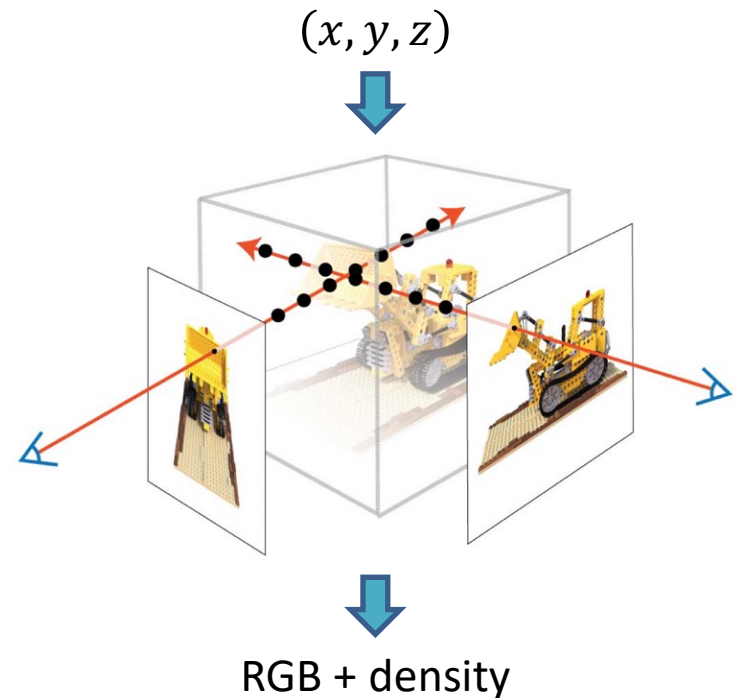
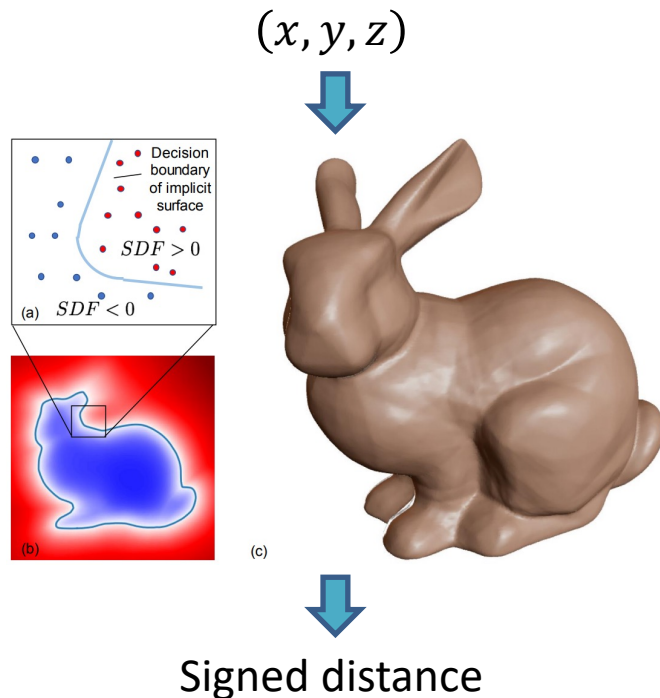
COLMAP



NeRF

Neural SDFs for Surface Reconstruction

Surface reconstruction vs. Volumetric reconstruction



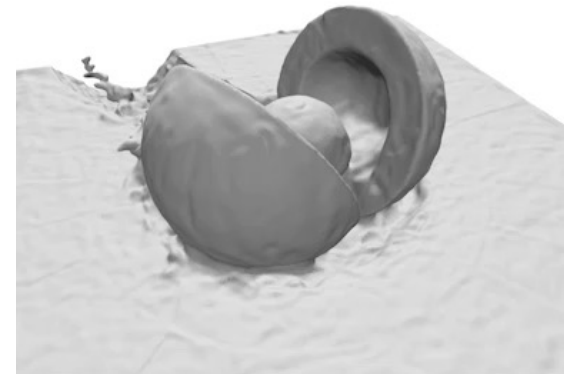
Neural SDFs for Surface Reconstruction



Reference image



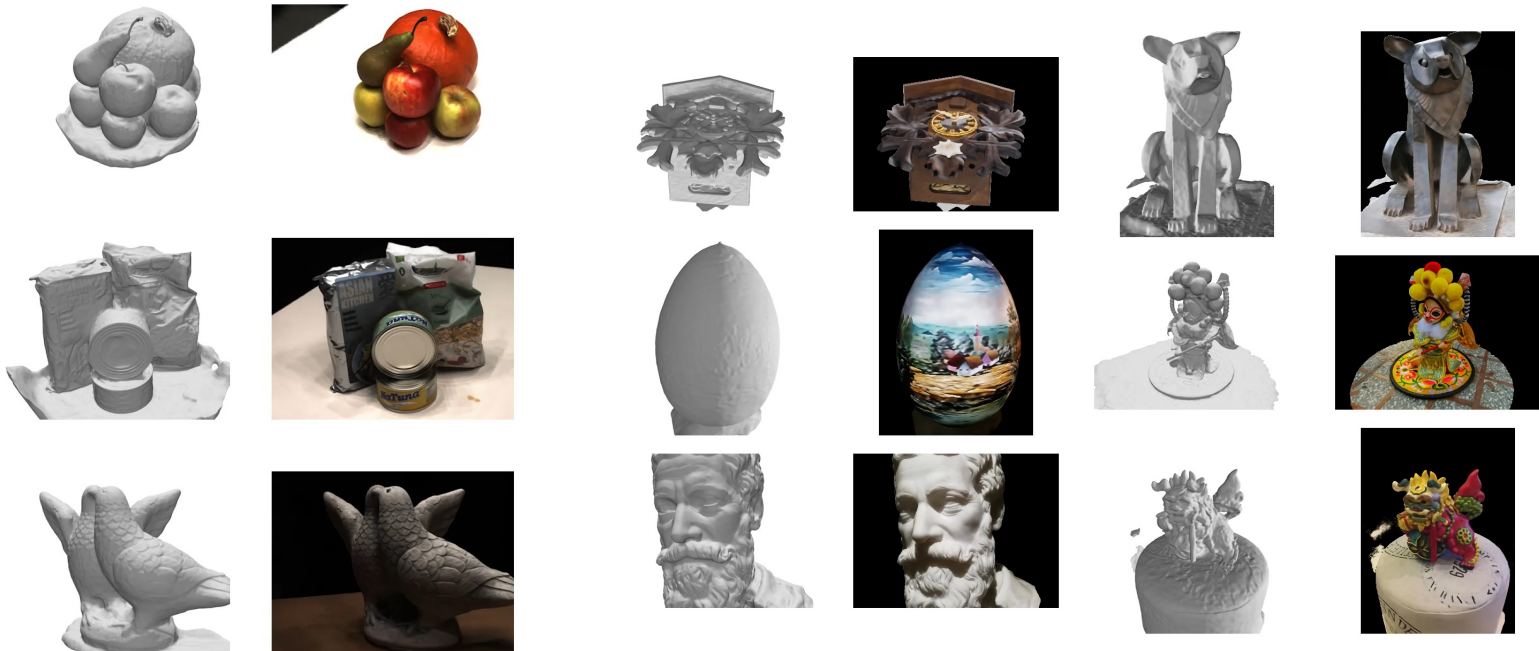
NeRF



NeuS

NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction, *NeurIPS* 2021.

NeuS



NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction, *NeurIPS* 2021.

Question?